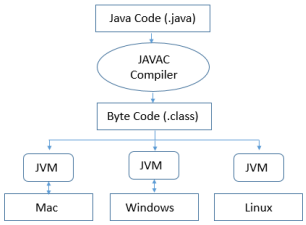
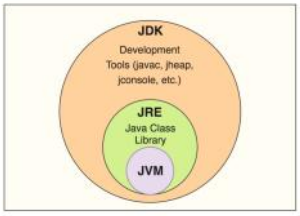
What is Java

29 July 2018 09:01

Introduction	Java is "**architecture-neutral, interpreted and portable**". Java programs are compiled to bytecode that have no dependencies on a specific machine architecture. **In order to run on a particular system, all you need is a Java Interpreter only which is JRE.** It's statically typed i.e data type of variables are declared before use. It's strongely typed i.e Variables have strict restrictions on the range and type of values they can hold like short, int, lo ng. 
Platform Independent	First, we write Java source file and compile it. When we **compile Java source code using javac**, the result is a **.class file that contains bytecode**. **The bytecode is the same no matter what platform we are on**, provided that we are using Java Virtual Machine (JVM) which is platform dependent in nature. **This JVM converts the byte code to machine code** according to the original computer's machine architecture like x86,ARM etc. Java Virtual Machine (JVM) is of different type, according to computer system architecture, that means for x86 JVM will be different for ARM JVM.
Components	**JVM**: It's an abstract machine that provides the runtime environment to execute Java bytecode by translating it into platform -specific native binary code. The JVM itself is **platform-dependent.** Whatever Java program we run, JVM is responsible for executing the java program line by line(Using JIT) hence it is also know n as interpreter. JVM is responsible for: • loading classes • verifying bytecode • managing memory • executing bytecode • garbage collection • JIT compilation • thread management **JRE** is an installation package which provides environment to only run(not develop) the java program(or application)onto the machi ne. JRE is only used by them who only wants to run the Java Programs(Byte Code/class file) i.e. end users of the system. It contains JVM, core libraries and it cannot co mpile code. After Java 9 separate JRE package was removed to reduce code duplication. Users can generate custom jre via jlink. **JDK** is a Kit that provides the environment to develop and execute(run) the Java program. It compiles .java to .class and jvm executes this byte code. It includes compiler(javac), runtime(java), Jshell, archiver(jar), doc generator etc. JDK,JRE and JVM are OS specific but bytecode is platform independent. 
Objects and Classes	**Object**: A real-world entity defined by two specific components • **State (Properties)**: Characteristics like color, age, or brand. • **Behavior (Functionality)**: Actions the object performs, such as bark(), drive(), or applyBrake(). **Class**: The blueprint, template, or skeleton used to create objects.
JIT	Java is both: • compiled • interpreted **JIT Characteristics** • converts bytecode → native machine code • does not compile entire application • compiles hot methods only • caches machine code • eliminates repeated interpretation • performs runtime optimization • Compiled code is stored in Native Memory Code Cache Examples of optimization: • method inlining • loop unrolling • escape analysis • dead code elimination • lock elision
Execution Formats Across Versions	• Prior to Java 11: Required explicit compilation using javac filename.java followed by execution using java filename like java Main.java and not • javac Main.java • java Main • Java 11 Single File Programs: Allows execution of single file programs directly without a separate compilation step by running java filename.java. • Java 22 Multi File Programs: Expands the direct execution feature to applications split across multiple source files without explicit pre-compilation. New features only reduce manual steps, not underlying process • Java 25 Compact Source Files: Introduces beginner friendly code reduction by allowing instance main methods to run without a formal class declaration wrapper. This is not replacement for standard syntax in production but Only a convenience feature. `void main() {`

```
        IO.println("Hello");
    }
```

Stack and Heap

30 July 2018 19:52

JVM Memory Management	JVM MEMORY | --- | | Shared Thread Local | | Heap Memory JVM Stack Method Area PC Register Native Method Stack
Stack	Stack memory is a region of the RAM that stores local variables created by each function. It also contains method specific va lues that are short-lived and **references to other objects in the heap that are getting referred from the method**. Every time a function declares a new variable, it is "pushed" onto the stack . When a function is called, a block is reserved on the top of the stack for local variables. Then every time a function exits, the block becomes unused and can be u sed the next time a function is called. The stack is always reserved in a last in first out (LIFO) order; the most recently reserved block is always the next block to be freed. T his makes it really simple to keep track of the stack; freeing a block from the stack is nothing more than adjusting one pointer. Stack memory stores method execution context inclu ding local variables and object references. Heap memory stores actual objects and instance data. When a method completes, its stack frame is removed, but objects remain in he ap until garbage collected. Objects become eligible for GC when no references point to them. **Summary** • The stack grows and shrinks as functions push and pop local variables • There is no need to manage the memory yourself, variables are allocated and freed automatically • Stack has size limits • Stack variables only exist while the function that created them, is running • **Each thread has its own stack** • When stack memory is full, stackoverflow error is thrown
Heap	Java Heap space is used by java runtime to allocate memory to Objects and JRE classes. Whenever we create any object, it's al ways created in the Heap space. Unlike the stack , there's no enforced pattern to the allocation and deallocation of blocks from the heap; you can allocate a block at any time and free it at any time. Any object created in the heap space has global access and can be referenced from anywhere of the program. Java Garbage Collector is an automatic memory man agement system that reclaims heap memory for objects. Heap Memory Stores • Objects • Arrays • String Constant Pool (since Java 7) • Instance variables • Static Variables `Student s = new Student(); s is stored in stack which points to student object in heap` **Heap Characteristics**: • Shared among threads • Managed by GC • Dynamically allocated • Largest JVM memory area • Memory Fragments over time • throws an OutOfMemoryError when full • Variables can be accessed globally • Relatively slower access **Types of References** **Strong Reference**: Person p = new Person(); • Default reference • GC **will NOT delete object** **Weak Reference**: WeakReference<Person> wp = new WeakReference<>(new Person()); • GC **can delete anytime** • Access after GC → returns **null** **Soft Reference**: • Similar to weak • GC removes **only when memory is low**
Heap Memory Structure	├── Young Generation │ ├── Eden │ ├── Survivor S0 │ └── Survivor S1 │ └── Old Generation **Young Generation**: Object Lifecycle 1. New object → **Eden** 2. Minor GC runs → moves survivors to **S0/S1** 3. Age increases each cycle 4. Alternate between S0 ↔ S1 Minor GC • Happens in **Young Generation** • Fast and frequent Promotion to Old Generation • Objects surviving multiple GC cycles: ○ Reach **threshold age** ○ Moved to **Old Generation** **Old Generation**: • Stores long-lived objects • GC here is: • **Major GC**

	• Slower • Less frequent
Program Counter Register	Each thread has a PC Register which Stores current instruction address being executed in order to resume execution after thre ad switching.

JVM Memory

Memory Area	Type	What it Stores	Managed By / Lifecycle
Heap	JVM Managed (On-Heap)	* All objects created with new. * Instance variables. * **Static variables** (inside java.lang.Class objects). * **String Constant Pool**.	**Garbage Collector (GC)**. Objects are cleared only when they become unreachable. System.gc() requests a run but offers no guarantee.
Stack	Thread Local (On-Heap)	* Method calls (Stack Frames). * Local variables and parameters. * Object reference pointers (e.g., the local variables).	**Thread Execution**. Automatically allocated when a method is invoked, and destroyed immediately when the method exits.
Native Memory	OS Allocated (Off-Heap)	* JIT compiled machine code cache. * JNI (Java Native Interface) allocations. * Off-heap Direct Buffers (e.g., ByteBuffer.allocateDirect()).	**JVM Internal Subsystems & Operating System**. Not managed by the standard Garbage Collector (must be manually freed or handled via JVM bridges).
Metaspace	**Native Memory** (Off-Heap)	* Class metadata and definitions. * Method bytecode. * Runtime constant pools.	**JVM ClassLoader**. Grows dynamically using OS RAM. Allocations are cleared only when their defining ClassLoader is unloaded.

```
public class Demo {
   public static void main(String[] args) {
      Employee e = new Employee();
   }
}
```

- Class loaded → **Metaspace**
- main() invoked → **Stack frame created**
- new Employee() → **Object in Heap**
- e holds reference → **Stack**
- Method ends → stack cleared
- Object → **eligible for GC**

Category	Element	Exact Storage Location	Lifecycle & Notes
Methods & Metadata	**Static Method Definitions** (Bytecode)	**Metaspace** (Native/Off-Heap)	Loaded once per class. Destroyed only if the ClassLoader is unloaded.
	Instance Method Definitions (Bytecode)	**Metaspace** (Native/Off-Heap)	Exactly like static methods; the actual code blueprint lives here.
	Class Metadata (Reflected structures)	**Metaspace** (Native/Off-Heap)	Contains the runtime constant pool, field definitions, and method tables.
Variables (Class & Instance Level)	**Static Variables** (Primitives & Object References)	**Java Heap** (Inside the java.lang.Class object)	Allocated when the class is initialized. Persists for the lifetime of the class.
	Instance Variables (Primitives & Object References)	**Java Heap** (Inside the specific object)	Created with new. Lives and dies alongside the parent object instance.
	Array Elements	**Java Heap**	Arrays are treated as full objects in Java, regardless of whether they hold primitives or references.
Execution (Thread-Level)	**Local Variables & Method Parameters**	**Thread Stack** (Inside a Stack Frame)	Created instantly when a method is called; destroyed immediately when the method exits.
	Reference Variables (The pointers themselves)	**Thread Stack** (If local) *or* **Java Heap** (If instance/static field)	The pointer itself lives where it's declared, but it always *points* to an object on the Heap.
Special Memory Structures	**String Constant Pool**	**Java Heap** (Special internal region)	Stores interned strings to optimize memory. Moved out of PermGen in Java 7+.
	JIT Compiled Code (Native Code Cache)	**Native Memory** (Code Cache)	Highly optimized machine code compiled at runtime by the JIT compiler.
	Direct Buffers (e.g., ByteBuffer.allocateDirect)	**Native Memory** (Off-Heap)	Bypasses Java Heap for ultra-fast I/O operations; not directly managed by standard GC.

Strings

08 June 2018 11:47

<table>
<tr><td>Definition</td><td>A String is a sequence of Unicode characters.

String name = "Saurav";

Internally, String is: public final class String

Characteristics:
• Immutable: to ensure String pool safety as changing one value might change other variable as well
• thread-safe
• serializable
• comparable</td></tr>
<tr><td>String str1="Hello";
String str2="World";</td><td>Here Hello and World are the objects and str1 and str 2 points to the objects</td></tr>
<tr><td>String str1="Hello";
String str2="Hello";</td><td>Here only one object Hello will be created in string constant pool and str1 and str2 will point to it. This is called String Literal Method and objects are created in String Constant Pool.</td></tr>
<tr><td>String Constant Pool</td><td>SCP is the space reserved in the heap memory that can be used to store the strings. The main advantage of using the String pool is whenever we create a string literal; the JVM checks the "string constant pool" first. If the string already exists in the pool, a reference to the pooled instance is returned. If the string doesn't exist in the pool, a new string instance is created and placed in the pool. Therefore, it saves the memory by avoiding the duplicacy.</td></tr>
<tr><td>String object with new keyword</td><td>String str1=new String("Welcome");
• SCP: Checks for "Welcome". It's not there, so it creates 1 object in the SCP.
• Heap: The new keyword creates 1 object in the heap.
• Total for Line 1: 2 objects created.

String str2=new String("Welcome");
• SCP: Checks for "Welcome". It is already present (from Line 1), so 0 objects are created here.
• Heap: The new keyword creates another 1 object in the heap.
• Total for Line 2: Only 1 object created.

Ultimately, str1 and str2 will point to two completely different objects in the heap, even though both of those heap objects internally point to the exact same "Welcome" literal in the SCP.

Because of this duplicate object creation in the heap, it is generally recommended to just use standard literals (String str = "Welcome";) unless you have a highly specific reason to force separate object identities.</td></tr>
<tr><td>Equality</td><td>== checks if both objects point to the same memory location whereas .equals() evaluates to the comparison of values in the objects</td></tr>
<tr><td>String Methods</td><td>
<table>
<tr><th>Method / Feature</th><th>Return Type / Category</th><th>Purpose & Behavior</th><th>Example Code & Output</th></tr>
<tr><td>indexOf()</td><td>int</td><td>Returns the position of the first occurrence of a character or substring. Returns -1 if not found.</td><td>String str = "hello world!";
str.indexOf("world"); // 6
str.indexOf("@"); // -1</td></tr>
<tr><td>lastIndexOf()</td><td>int</td><td>Returns the position of the last occurrence of a substring.</td><td>String str2 = "abc world xyz world";
str2.lastIndexOf("world"); // 14</td></tr>
<tr><td>contains()</td><td>boolean</td><td>Checks if a specific substring exists inside the string.</td><td>String str = "Java is fun!";
str.contains("Java"); // true</td></tr>
<tr><td>startsWith()</td><td>boolean</td><td>Checks if the string begins with the specified prefix.</td><td>String str = "Java is fun!";
str.startsWith("Java"); // true</td></tr>
<tr><td>endsWith()</td><td>boolean</td><td>Checks if the string ends with the specified suffix.</td><td>String str = "Java is fun!";
str.endsWith("fun!"); // true</td></tr>
<tr><td>matches()</td><td>boolean</td><td>Performs a pattern-based search matching the entire string against a regular expression.</td><td>String str = "hello world";
str.matches(".*world.*"); // true</td></tr>
<tr><td>valueOf()</td><td>String</td><td>Converts primitive data types (integers, characters, char array etc.) or objects into their String representations.</td><td>String.valueOf(123); // "123"
String.valueOf('A'); // "A"
char[] arr = {'J', 'a', 'v', 'a'};
String str = String.valueOf(arr); // "Java"</td></tr>
<tr><td>charAt()</td><td>char</td><td>Returns the character located at the specified index.</td><td>"Java".charAt(0); // 'J'</td></tr>
<tr><td>isEmpty()</td><td>boolean</td><td>Checks if the string length is 0. Using "".equals(str) is preferred for safety as it prevents a NullPointerException if the string is null.</td><td>String myStr = "";
myStr.isEmpty(); // true
"".equals(myStr); // true</td></tr>
<tr><td>trim()</td><td>String</td><td>Removes leading and trailing whitespace characters from the beginning and end of a string.</td><td>String str = " hello ";
str.trim(); // "hello"</td></tr>
<tr><td>substring()</td><td>String</td><td>Extracts a portion of a string. The start index is inclusive, and the end index is exclusive.</td><td>String orig = "Hello, World!";
orig.substring(7); // "World!"
orig.substring(0, 5); // "Hello"</td></tr>
<tr><td>split()</td><td>String[]</td><td>Splits a string into an array based on matching criteria.
• \\s+: Splits by any whitespace sequence, ignoring duplicates.
• " ": Splits by single spaces, keeping empty strings for consecutive spaces.
• "": Splits into individual characters.</td><td>String text = "apple,banana,orange";
text.split(","); // ["apple","banana","orange"]

"Hello World".split(" ");
// ["Hello", "", "World"]</td></tr>
<tr><td>String.format()</td><td>String</td><td>Formats a string using placeholders (%s for strings, %d for integers, %f for floating-points).</td><td>String.format("Name: %s, Age: %d", "John", 25);
// "Name: John, Age: 25"</td></tr>
<tr><td>System.out.printf()</td><td>void</td><td>Directly prints a formatted string to the console using placeholders (avoids manual string creation).</td><td>System.out.printf("Price: %.2f%n", 19.99);
// Prints: Price: 19.99</td></tr>
<tr><td>Integer.parseInt()</td><td>int</td><td>Parses a string argument as a signed decimal integer.</td><td>int num =</td></tr>
</table>
</td></tr>
</table>

				Integer.parseInt("456"); // num = 456
	replace(char, char)	String	Replaces all instances of a matching literal character. Creates a new object because Strings are immutable.	String str = "Hello, World!"; str.replace('o', '*'); // "Hell*, W*rld!"
	replace(CharSequence, CharSequence)	String	Replaces all literal occurrences of a matching substring target sequence.	"abababab".replace("ab", "X"); // "XXXX"
	replaceAll()	String	Replaces all matching substrings that satisfy the specified regular expression.	"Java is fun!".replaceAll("a|e|i|o|u", "*"); // "J*v* *s f*n!"
	replaceFirst()	String	Replaces only the very first match found satisfying a regular expression.	"apple orange apple".replaceFirst("apple", "grape"); // "grape orange apple"
	String.join()	String (Static)	Joins multiple elements or collections together using a delimiter. Internally leverages StringBuilder.	List<String> words = Arrays.asList("A", "B", "C"); String.join("-", words); // "A-B-C"
	Text Blocks	Syntax Feature	Multi-line string syntax (""") introduced in Java 15. Double quotes don't need escaping. • \ at end of line: Prevents newline. • \s: Preserves trailing spaces.	String json = """ { "user": "Gemini" } """;
	toCharArray()	Char[]	Converts String to an array of characters	Str.toCharArray();

String Interning

String Interning in Java is a process of storing only one copy of each distinct String value, which must be immutable.

```
String hello = "hello";
String obj = new String("hello").intern();
System.out.println(hello==obj);//true
```

String Builder and Buffer(Thread Safe)

• Stored as **resizable char array and** Capacity grows dynamically.
• StringBuffer sb = new StringBuffer("Hello");

append(String str)	sb.append("Hello");
insert(int offset, String str)	sb.insert(1, "Java");
delete(int start, int end):	sb.delete(1, 3);
deleteCharAt(int index):	sb.deleteCharAt(2);
reverse()	sb.reverse();
setCharAt(int index, char ch):	sb.setCharAt(1, 'J');
IndexOf, lastIndexOf	indexOf(String.valueOf('c'))

Character Functions

Boolean Functions

- **isLetter(char ch)**: Returns true if the character is a standard linguistic letter (uppercase, lowercase, titlecase, modifier, or other letter).
- **isDigit(char ch)**: Returns true if the character is a decimal digit (0 through 9).
- **isLetterOrDigit(char ch)**: A convenience method that returns true if the character is either a letter or a digit.
- **isAlphabetic(int codePoint)**: Returns true if the character is a letter or a "letter-like" symbol (including Roman numerals and certain combining marks).
- **isWhitespace(char ch)**: Returns true if the character is a Java whitespace character (includes space, tab, newline, carriage return, etc.).
- **isUpperCase(char ch) / isLowerCase(char ch)**: Checks if the character is specifically an uppercase or lowercase letter according to Unicode.
- **isSpaceChar(char ch)**: Returns true if the character is a Unicode space character (a narrower check than isWhitespace, as it focuses strictly on s pace-type characters rather than control characters like \n).
- **isDefined(char ch)**: Returns true if the character has a defined meaning in Unicode (i.e., it isn't an unassigned "reserved" code point).

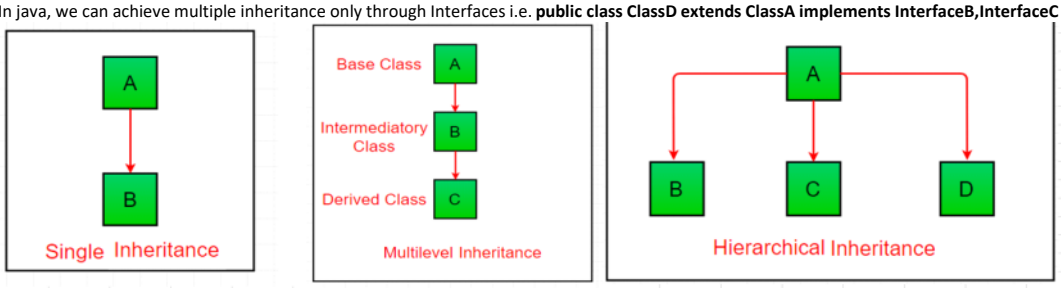
Transformation & Value Functions

- **toUpperCase(char ch) / toLowerCase(char ch)**: Returns the uppercase or lowercase equivalent of the character. If no equivalent exists, it returns the original character.
- **getNumericValue(char ch)**: Returns the int value that the character represents. For example, '5' returns 5, and 'A' (hexadecimal) returns 10.
- **compare(char x, char y)**: Compares two char values numerically; returns 0 if they are equal, a negative value if x < y, and a positive value if x > y .
- **toString(char ch)**: Converts the individual char into a String object containing that single character.
- **valueOf(char ch)**: Returns a Character instance (the object wrapper) representing the specified char value.

Inheritance

30 July 2018 15:46

<table>
<tr><td>Introduction</td><td>It is the mechanism in java by which one class is allowed to inherit the features(fields and methods) of another class.</td></tr>
<tr><td>Terminology</td><td>• Super Class: The class whose features are inherited is known as super class(or a base class or a parent class).
• Sub Class: The class that inherits the other class is known as sub class(or a derived class, extended class, or child class). The subdass can add its own fields and methods in addition to the superclass fields and methods.</td></tr>
<tr><td>Reusability</td><td>Inheritance supports the concept of “reusability”, i.e. when we want to create a new class and there is already a class that includes some of the code that we want, we can derive our new class from the existing class. By doing this, we are reusing the fields and methods of the existing class.

Constructors and initialisers are not inherited.</td></tr>
<tr><td>Types</td><td>• Single Inheritance: Cucumber Runner
• Hierarchical: Testbase and its sub classes
• Java does not support multiple inheritance with classes.
• In java, we can achieve multiple inheritance only through Interfaces i.e. public class ClassD extends ClassA implements InterfaceB,InterfaceC
 
</td></tr>
<tr><td>IS-A Relationship</td><td>Achieved through inheritance, like Dog is-a animal.</td></tr>
<tr><td>HAS-A Relationship</td><td>Association is the most general term for a relationship between two separate classes that is established through their objects. It defines how objects "use" each other. It can be one-to-one, one-to-many or many-to-many.

<table>
<tr><td>Aggregation</td><td>Aggregation is a "weak" association. It represents a relationship where the child can exist independently of the parent.
• Relationship: Often called a HAS-A relationship.
• Life Cycle: If the parent object is destroyed, the child object can still survive.
• Example: A BasePage aggregating the WebDriver. If the Page Object is garbage collected, the browser (driver) stays open.</td></tr>
<tr><td>Composition</td><td>Composition is a "strong" association. It represents a relationship where the child cannot exist independently of the parent.
• Relationship: A more intense version of HAS-A.
• Life Cycle: If the parent object is destroyed, the child objects are also destroyed.
• Automation Example: A LoginPage class HAS-A Button (Submit) or TextBox (Username). If the LoginPage object is destroyed, those specific element definitions on that page go away too.</td></tr>
</table></td></tr>
<tr><td>Composition over Inheritance</td><td>In modern Java development, it is often recommended to "favor composition over inheritance." This makes your code more flexible and easier to change because you aren't locked into a rigid class hierarchy.

Composition is generally preferred over inheritance in enterprise automation frameworks because it promotes loose coupling and high cohesion.

With inheritance: public class LoginPage extends BaseDriver

LoginPage automatically inherits all behaviors and state from BaseDriver, even if it only needs WebDriver access. This creates strong coupling and makes framework evolution difficult.

With composition:

public class LoginPage {
  private WebDriver driver;
}

the page depends only on the capability it requires.

Benefits:
• Maintainability: changes in driver management don't cascade across hundreds of page classes.
• Loose Coupling and High Cohesion: pages depend on interfaces/capabilities rather than inheritance hierarchy.
• Extensibility: driver implementation can be replaced without modifying page classes.
• Testability: dependencies can be mocked and injected.
• Enterprise scalability: avoids fragile inheritance trees that become difficult to maintain at scale.

In large frameworks, inheritance should be used sparingly for shared contracts, while composition should be the default mechanism for behavior reuse.

Prefer composition + private state over protected inheritance hierarchies.</td></tr>
<tr><td>Sealed Class/Interface</td><td>A class or interface that uses the sealed modifier to limit its subclasses. It must specify its allowed subclasses using the permits clause.

Permitted Subclasses: The specific classes allowed to extend the sealed parent. They must declare whether they are final (cannot be subclassed further), sealed (subclassed with their own restrictions), or non-sealed (opened back up to regular inheritance).

<pre>// 1. Define the sealed interface and permit only two specific implementations

public sealed interface Account permits CheckingAccount, SavingsAccount {
    String getAccountType();
}

// 2. A permitted subclass can be 'final' (closes the inheritance chain)

public final class CheckingAccount implements Account {
    @Override</pre></td></tr>
</table>

```
    public String getAccountType() {
        return "Checking Account";
    }
}

// 3. Or a permitted subclass can be 'non-sealed' (allows anyone to extend it)

public non-sealed class SavingsAccount implements Account {
    @Override
    public String getAccountType() {
        return "Savings Account";
    }
}
```

Abstraction

30 July 2018 17:45

Abstraction is the process of hiding implementation details and exposing only functionality. In other words, the user will ha ve the information on what the object does instead of how it does it.

Why WebDriver is an interface/ Meaning of Abstraction
The purpose of Selenium WebDriver to automate browsers but they do not know how each browser works internally. Every browser has their own logic to perform browser's actions such as Launching a browser, closing a browser, loading URL, handling different type of web elements. Same operations are performed in differen t ways by different browsers.

Without knowing internal working mechanisms of browsers, Selenium WebDriver developers can not write codes to perform actions . If they get to know the internal mechanism some how, it will be difficult to manage when internal mechanism changes and this should not break flow for other browsers. So to make it simple, a contract is placed between Selenium WebDriver developers and browsers and that contract is in form of WebDriver. A WebDriver consists of all related basic methods which could be performe d on a browser.

Feature	**Abstract Class** (The "is-a"/identity relationship)	**Interface** (The "can-do"/Ability relationship)
Definition	A class that may or may not contain abstract methods. It can provide a common base for related classes.	A blueprint of a class, defining a contract for what a class *must* do.
Relationship	IS-A	CAN-DO
Purpose	Identity	Capability
Constructors	Yes	No
Instance variables	Yes	No
Multiple inheritance	No	Yes
Abstract methods	Yes	Yes
Concrete methods	Yes	Yes
Static methods	Yes	Yes
Default methods	Yes	Yes
Private methods	Yes	Yes (Java 9+)
State	Can store	Cannot store
Variables	Can have any type of variables (static, non-static, final, non-final).	All variables are implicitly public, static, and final (constants). Follow Snake Case Naming convention i.e UNIVERSE_NAME
Instantiation	Cannot be instantiated directly.	Cannot be instantiated directly.
Inheritance	Subclasses use the extends keyword to inherit.	Classes use the implements keyword to implement.
Purpose	To provide a common base class and code reuse for related classes.	To define a contract or specification for classes to follow.
Use Case	Used "When common code is shared." When you have a hierarchy of **related classes** that need to share a base state or common behavior. Provides a **template**. It says: "We are all the same 'type' of thing, so here is some shared code." **AbstractList** class: It is inherited by ArrayList and LinkedList. get() is an abstract method in AbstractList.The AbstractList doesn't know how your data is stored (is it an array? a chain of nodes?). Therefore, it marks get(int index) as abstract. It says: "I know a list must have a get method, but ArrayList and LinkedList, you have to write the logic yourselves."	Used "When multiple implementations are expected." When you want to define a role or capability that multiple unrelated classes can share. Imagine an interface called Encryptable. This is a "role" or a "capability." • **Unrelated Class A:** UserPassword (A string of text) • **Unrelated Class B:** BankTransaction (A financial record) • **Unrelated Class C:** EmailMessage (A communication object) These three things have **no common parent**. A password is not a transaction. However, they all need the **ability** to be encrypted. By implementing the Encryptable interface, they all agree to provide a hideData() method.

Encapsulation

30 July 2018 18:02

Definition:

- Wrapping data and methods into a single unit (a class).
- A protective shield preventing external access to data using private data members.

Technical Aspects:

- Data (variables) is hidden (private).
- Access to data is controlled through public methods (getters and setters).

Advantages:

- **Data Hiding:** Protects data from unintended changes.
- **Improved Maintenance:** Changes to internal implementation don't affect external code.
- **Modularity:** Breaks complex systems into manageable units.
- **Reusability:** Encapsulated code can be used in different parts of an application.
- **Flexibility:** Internal implementation can be changed without altering the external interface.
- **Simplified Testing & Debugging:** Individual components can be tested and debugged easily.
- **Increased Security:** Controls access to sensitive data.

Key takeaway: Encapsulation is about bundling data and methods together and controlling access to that data, leading to better code organiz ation, maintainability, and security. e.g. pageObject Model

Polymorphism

30 July 2018 18:09

Method Overloading	Method Overloading is a feature that allows a class to have more than one method having the same name, if their argument list s are different. **It can't be achieved via different return type. Private, static and final can't be overridden**. The access modifier of overriding class cannot be more restricting then overridden method. Its also called compile time polymorphism or static binding.
Example	**Function Overloading in Selenium**: driver.switchTo().frame("frameName/ID"); driver.switchTo().frame(1); driver.switchTo().frame(webElement); Action.click(); Action.click(WebElement); **Constructor Overloading in Selenium:** WebDriver driver = new ChromeDriver(); WebDriver driver = new ChromeDriver(options);
Method Overriding	Overriding is a feature that allows a subclass or child class to provide a specific implementation of a method that is alread y provided by one of its super-classes or parent classes. When a method in a subclass has the same name, same parameters or signature and same return type(or sub -type) as a method in its super-class, then the method in the subclass is said to override the method in the super-class. ChromeDriver class overrides get() of RemoteWebDriver class.
Example	It is the type of the object being referred to (not the type of the reference variable) that determines which version of an o verridden method will be executed. class Animal { public void move() { System.out.println("Animals can move"); } } class Dog extends Animal { public void move() { System.out.println("Dogs can walk and run"); } public void bark() { System.out.println("Dogs can bark"); } } public class TestDog { public static void main(String args[]) { Animal a = new Animal(); // Animal reference and object Animal b = new Dog(); // Animal reference but Dog object (Top/Upcasting) a.move(); // runs the method in Animal class b.move(); // runs the method in Dog class (Dynamic Polymorphism) b.bark(); // Error } } Compile time error as bark is missing in animal. In compile time, the check is made on the reference type. However, in the ru ntime, JVM figures out the object type and would run the method that belongs to that particular object. TestDog.java:26: error: cannot find symbol b.bark(); ^ symbol: method bark() location: variable b of type Animal 1 error
Shadowing/Hiding of static functions	This is not overloading. If a subclass defines a static method with the same signature as a static method in the super class then the method in the su bclass hides the one in the super class.This happens because the static method is resolved at the compile time. Static methods bind during the compile time using the type of reference not a type of object. // file name: Main.java class A { static void fun() { System.out.println("A.fun()"); } } class B extends A { static void fun() { System.out.println("B.fun()"); } } public class Main { public static void main(String args[]) { A a = new B(); a.fun(); // prints A.fun() } } If we make both A.fun() and B.fun() as non-static then the above program would print “B.fun()”.

Constructors

30 July 2018 11:15

A constructor is a special member of a class used to initialize an object. Like methods, a constructor also contains collection of statements that are executed at time of Object creation. The purpose of the default constructor is to assign the default value to the objects. The java compiler creates a default constructor implicitly if there is no constructor in the class. The access level of default constructor is same as it's Class.

Car a = new Car(); Here **new Car() is object** and **a is object reference variable of type Car**. New keywork tells compiler to call contstructor of car class(car class may have car method as well)

Characteristics:

- Has the same name as the class.
- Has no return type (not even void).
- Invoked automatically when an object is created.
- Used to initialize object state.
- Can be overloaded.
- Cannot be inherited.
- Cannot be overridden.
- Can have access modifiers.
- Every class has a constructor whether it's a normal class or an abstract class.
- Constructors are not methods and they don't have any return statement(in order to distinguish between constructor and method of same name) but **constructor returns current class instance**.
- Constructor can use any access specifier, they can be declared as private also. Private constructors are possible in java but there scope is within the class only.
- **this() and super() should be the first statement in the constructor code.** If you don't mention them, compiler does it for you accordingly as parent object construction must complete before child object construction begins.
- A constructor can also invoke another constructor of the same class – By using this()/super().
- this(5) inside another constructor will call a parameterized constructor and pass value 5 to it.
- A constructor in Java cannot be abstract, final(Constructors cannot be inherited and final prevents overriding in sub class), static and Synchronized.
- Types:Param,Non Pram and Default Constructors
- If a class has a custom constructor, then compiler doesn't add default constructor automatically.
- **Constructor Chaining**:

```
Employee() {
    this("Default", "User");
}

Employee(String f, String l) {
    this.firstName = f;
    this.lastName = l;
}
```

- Execution order:
    1. Parent static
    2. Child static
    3. Parent instance
    4. Parent constructor
    5. Child instance
    6. Child constructor

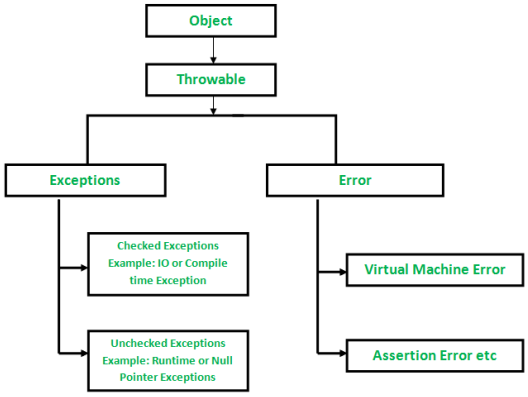
Access Modifier

29 July 2018 10:31

Modifier	public	protected	default	private
Same class	Yes	Yes	Yes	Yes
Same Package subclass	Yes	Yes	Yes	No
Same Package non-subclass	Yes	Yes	Yes	No
Different package subclass	Yes	Yes	No	No
Different package non-subclass	Yes	No	No	No

Exceptions

29 July 2018 12:28

<table>
<tr><td>Definition</td><td>An exception is an unwanted or unexpected event, which occurs during the execution of a program i.e at run time, that disrupts the normal flow of the program’s instructions.
• Error: An Error indicates serious problem that a reasonable application should not try to catch. OutOfMemoryError, VirtualMachineError, AssertionError etc.
• Exception: Exception indicates conditions that a reasonable application might try to catch.</td></tr>
<tr><td>Exception Hierarchy</td><td>All exception and errors types are sub classes of class Throwable, which is base class of hierarchy.
• One branch is headed by Exception. This class is used for exceptional conditions that user programs should catch. NullPointerException is an example of such an exception.
• Another branch, Error are used by the Java run-time system(JVM) to indicate errors having to do with the run-time environment itself(JRE). StackOverflowError is an example of such an error.
 
</td></tr>
<tr><td>How JVM Handles an Exception</td><td>• A method creates an Exception Object containing the exception's name, description, and program state.
• This object is "thrown" and handed to the JVM.
• The JVM searches the Call Stack (the list of methods called) in reverse order for an appropriate exception handler (a method that can handle that type of exception).
• If a matching handler is found, the exception is passed to it.
• If no handler is found, the JVM's default exception handler prints the exception details (name, description, and call stack) and terminates the program abnormally.
• Exception in thread "xxx" Name of Exception : Description ... . // Call Stack</td></tr>
<tr><td>Catch multiple exceptions</td><td>A method can throw more than one exceptions. However that method needs to declare all the checked exceptions it can throw

try {
//Do some processing which throws exceptions
} catch(SQLException | IOException e) {
someCode();
} catch(Exception e) {
someCode();
}

In this feature, you can catch multiple exceptions in single catch block. Before java 7, it was restricted to catch only one.

The order of catch execution is whatever matches first, gets executed. If the first catch matches the exception, it executes, if it doesn't, the next one is tried and on and on until one is matched or none are. So, when catching exceptions you want to always catch the most specific first and then the most generic (as RuntimeException or Exception).</td></tr>
<tr><td>Checked vs Unchecked Exceptions</td><td>1) Checked: are the exceptions that are checked at compile time. If some code within a method throws a checked exception, then the method must either handle the exception or it must specify the exception using throws keyword. For example, consider the following Java program that opens file at location “C:\test\a.txt” and prints first three lines of it. The program doesn’t compile, because the function main() uses FileReader() and FileReader() throws a checked exception FileNotFoundException. It also uses readLine() and close() methods, and these methods also throw checked exception IOException.

2)Unchecked are the exceptions that are not checked at compiled time. In Java exceptions under Error and RuntimeException classes are unchecked exceptions, everything else under throwable is checked. Division by 0 program compiles fine, but it throws ArithmeticException when run. The compiler allows it to compile, because ArithmeticException is an unchecked exception.</td></tr>
<tr><td>Custom Exception</td><td>Create a custom class inheriting Exception and in the constructor accept the error message and pass it to super constructor.

<table>
<tr><td>Define the Exception Class</td><td>Create a new class that extends one of the built-in exception classes:

• Exception: For general-purpose exceptions.
• RuntimeException: For unchecked exceptions.
• Other specific exception classes like IOException, SQLException, etc., for more specialized exceptions.

public class MyCustomException extends Exception {
// Constructor to initialize the exception message
public MyCustomException(String message) {
super(message);
}
}</td></tr>
<tr><td>Throw the Exception</td><td>public void myMethod() throws MyCustomException {
// ...
if (/* some condition is not met */) {
throw new MyCustomException("Custom exception message");
}
// ...
}</td></tr>
<tr><td>Catch and Handle the Exception</td><td>try {
myMethod();
} catch (MyCustomException e) {
System.out.println("Caught a custom exception: " + e.getMessage());
// Handle the exception, e.g., log the error, display an error message, etc.
}</td></tr>
<tr><td>Example</td><td>public class InvalidInputException extends Exception {
public InvalidInputException(String message) {
super(message);</td></tr>
</table>
</td></tr>
</table>

```
    }
}

public class MyClass {
    public void validateInput(int input) throws InvalidInputException {
        if (input < 0) {
            throw new InvalidInputException("Input cannot be negative");
        }
    }
}
```

Throw Vs Throws

throw keyword	throws keyword
The throw keyword is used to throw an exception explicitly.	The throws keyword is used to declare an exception.
The checked exceptions cannot be propagated with throw.	The checked exception can be propagated with throws
The throw keyword is used within the method.	The throws keyword is used with the method signature.
You cannot throw multiple exceptions.	You can declare multiple exceptions, e.g., public void method()throws IOException, SQLException.

Finally

finally executes when:
- exception occurs
- no exception occurs
- exception handled
- exception not handled

finally may NOT execute when: System.exit(0); or JVM crash occurs.

try-with-resources

Automatically closes resources.
Example:

```
try(FileReader fr =new FileReader("a.txt")){

}
```

Compiler generates:

```
finally{
    fr.close();
}
```

Benefits
- no resource leak
- cleaner code
- automatic cleanup
- Catch and finally are optional

Any class implementing AutoCloseable can be used. Example:
- Connection
- FileReader
- BufferedReader
- Scanner

Comparator & Comparable

03 December 2023 18:24

Comparator and Comparable are interfaces used for sorting objects in collections.

Comparator(Multi): It is an interface in java used to order the objects of user defined classes. It is useful when we need to provide multiple o rdering options of the objects.

Comparable: It is an interface in java that is used to define the natural ordering of objects for a user defined class. it is part of Jav a.lang package and it provides a compareTo method to compare instance of the class. A class has to implement a Comparable interface to define its natural ordering.

compareTo()	• It compares strings lexicographically(Dictionary Order) using Unicode values, character by character. • Return an **integer result** instead of boolean
Logic	**Return Value** / **Meaning** 0 / Both strings are equal < 0 / First string is smaller > 0 / First string is greater
Example	String a = "Java"; String b = "Python"; int result = a.compareTo(b); // negative value as J comes before P
Internal Working	Comparison happens **character by character** using **Unicode values**. "Java" vs "Python" J → Unicode 74 P → Unicode 80 74 - 80 = -6 1. Compare first characters 2. If different → return difference 3. If same → move to next character 4. Repeat until: • difference found OR • string ends 5. comparison depends on **first mismatch**
compareToIgnoreCase()	"java".compareToIgnoreCase("JAVA") // 0
Example	List<String> list = Arrays.asList("Python", "Java", "C"); Collections.sort(list); Collections.sort(list, Collections.reverseOrder());

Feature	Comparator	Comparable
Purpose	Defines custom ordering logic for objects.	Defines the natural ordering of objects.
Implementation	Implemented as a separate class or using lambda expressions.	Implemented by the class of objects being compared.
Method	compare(Object o1, Object o2): Compares two objects and returns a negative, zero, or positive integer.	compareTo(Object o): Compares the current object with another object and returns a negative, zero, or positive integer.
Sorting Behavior	Provides multiple sorting sequences based on different criteria.	Provides a single, inherent sorting sequence.
Class Modification	Does not modify the original class; ordering is external.	Modifies the original class to define its natural ordering.
Package	java.util	java.lang
Usage Scenario	When you need to sort objects based on criteria other than their natural ordering, or when the objects don't implement Comparable.	When you want to define the default, natural way objects of a class should be ordered.
Sorting Method Call	Collections.sort(List, Comparator) or Arrays.sort(array, Comparator)	Collections.sort(List) or Arrays.sort(array)
Flexibility	Highly flexible; allows for on-the-fly sorting logic changes.	Less flexible; requires modifying the class for any change in ordering.
Reusability	Can be reused for different collections of the same type or different types (if the logic applies).	Tied to the specific class that implements it.
Impact on Existing Code	Adding a new Comparator does not affect existing code that relies on the natural ordering.	Implementing Comparable affects all code that relies on the class's ordering.
Use with Anonymous Classes/Lambdas	Often used with anonymous classes or lambda expressions for concise, inline sorting logic.	Typically implemented directly within the class definition.
Null Handling	Can handle null values with custom logic within the compare() method.	compareTo() may throw NullPointerException if not handled within the implementation.
Example	@Data @AllArgsConstructor public class Student { private int rollNo; private String name; private Double averagePercentage; *public static Comparator<Student> rollComparator=(s1,s2)-> Double.compare(s1.rollNo,s2.rollNo);* *public static Comparator<Student> nameComparator=(s1,s2)-> s1.name.compareTo(s2.name);* Comparator<Student> sortByName=Comparator.*comparing*(Student::getName); Comparator<Student> sortByRno = Comparator.comparing(Student::getRno).reversed(); public static void main(String[] args) { List<Student> students = new ArrayList<>(); students.add(new Student(3, "Alice", 85.5));	@Data @AllArgsConstructor public class Student implements Comparable<Student> { private int rollNo; private String name; private Double averagePercentage; @Override public int compareTo(Student otherStudent) { //return Integer.compare(this.rollNo, otherStudent.rollNo); //return this.name.compareTo(otherStudent.name); return Double.compare(this.averagePercentage, otherStudent.averagePercentage); } public static void main(String[] args) { List<Student> students = new ArrayList<>();

```
        students.add(new Student(4, "Dick", 55.5));
        students.add(new Student(1, "Bob", 91.2));
        students.add(new Student(2, "Charlie", 78.9));

        //students.sort(rollComparator);
        students.sort(nameComparator);

        for (Student student : students) {
            System.out.println(student);
        }
    }

}
```

```
    students.add(new Student(3, "Alice", 85.5));
    students.add(new Student(1, "Bob", 91.2));
    students.add(new Student(2, "Charlie", 78.9));

    Collections.sort(students);

    for (Student student : students) {
        System.out.println(student);
    }
}}
```

Examples

Compare Strings	*public static Comparator<Student> nameComparator=(s1,s2)-> s1.name.compareTo(s2.name);*
Compare Numbers	*public static Comparator<Student> rollComparator=(s1,s2)-> Double.compare(s1.rollNo,s2.rollNo);*

Compare Strings	return this.name.compareTo(otherStudent.name);
Compare Numbers	return Double.compare(this.averagePercentage, otherStudent.averagePercentage);

- Achieve Sorting when section is same for 2 students:
  studentList.sort(sectionComparator.thenComparing(nameComparator));
- Sorting with reverse order:
  students.stream().sorted(nameComparator.reversed()).forEach(System.out::println);

Functional Programming

08 January 2021 10:16

<table>
<tr><td>Functional interfaces</td><td>can contain:
• one abstract method
• multiple default methods
• multiple static methods
• private methods (Java 9+)</td></tr>
<tr><td>Lambda</td><td>A lambda is an anonymous function.

A lambda expression(another way to define a method) is a short block of code which takes in parameters and returns a value. Lambda expressions are similar to methods, but they do not need a name, no access modifier and they can be implemented right in the body of a method without explicitly stating return type.

Lambda Restriction on Local Variables
1. Local variable cannot be used as lambda parameter
2. Local variables cannot be reassigned a value inside lambda(Local variables are effectively final inside lamdba)</td></tr>
<tr><td>Stream</td><td>Java provides a new additional package in Java 8 called java.util.stream. This package consists of classes, interfaces and enum to allows functional-style operations on the elements. You can use stream by importing java.util.stream package.

Stream provides following features:
• Stream does not store elements. It simply conveys elements from a source such as a data structure through a pipeline of computational operations.
• Stream is functional in nature. Operations performed on a stream does not modify its source. For example, filtering a Stream obtained from a collection produces a new Stream without the filtered elements, rather than removing elements from the source collection.
• Stream is lazy and evaluates code only when required.
• The elements of a stream are only visited once during the life of a stream. Like an Iterator, a new stream must be generated to revisit the same elements of the source.
• You can use stream to filter, collect, print, and convert from one data structure to other etc.
• parallelStream() enables parallel execution of stream operations. Instead of processing elements sequentially, tasks are divided into multiple smaller tasks that can run concurrently on different threads, leveraging the multi-core architecture of modern CPUs.</td></tr>
<tr><td>Method Reference</td><td>Each time when you are using lambda expression to just referring a method, you can replace your lambda expression with method reference.

System.out::println
Objects::isNull
String::toUpperCase
String::concat
String::replaceAll
list::add
Cat::new//constructor reference</td></tr>
<tr><td>Intermidiate Operations</td><td>Steam can have multiple intermediate operations and only 1 terminal operation as intermediate returns new stream while terminal consumes stream.

• Filter
• Map
• Limit
• Distinct
• Sorted</td></tr>
<tr><td>Terminal Operations</td><td>• Foreach
• Collect
• Count: returns long
• Min: returns optional
• Max: returns optional
• Findany: returns optional
• Findfirst: returns optional
• Anymatch: returns boolean
• Nonematch: returns boolean
• reduce</td></tr>
<tr><td>Flatmap</td><td>The flatMap method is similar to the map function in Java Streams, but with a significant difference: while map transforms each element in the stream into another form (one-to-one mapping), flatMap transforms each element into a stream of elements and then flattens these streams into a single stream. This is particularly useful when each element in the stream represents a collection (like a list or an array).

Flatmap returns stream of objects

List<String> studentActivities = StudentDataBase.getAllStudents()
.stream()
.map(Student::getActivities) // Stream<List<String>>
.flatMap(List::stream) // <Stream<String>
.collect(Collectors.toList());

Here flatmap is required because map is returning List of String, map cannot capture that hence flatmap is required to flatten it to string.</td></tr>
<tr><td>Optional</td><td>Java introduced a new class Optional in jdk8. It is a public final class and used to deal with NullPointerException in Java application. You must import java.util package to use this class. It provides methods which are used to check the presence of value for particular variable.

• Optional.ofNullable(value): Use when the value might be null. Returns an empty Optional if value is null.
• Optional.of(value): Use when you are sure the value is non-null. Throws NullPointerException if value is null.
• Optional.empty(): Use to represent an absent value.
• Optional.orElse(value): Returns the provided value if the Optional is empty.
• Optional.orElseGet(Supplier): Returns the value supplied by the Supplier if the Optional is empty.
• Optional.orElseThrow(Supplier): Throws an exception provided by the Supplier if the Optional is empty.
• The Optional.isPresent() method checks if there is a value inside the Optional. It returns true if the Optional contains a value, otherwise false.
• The Optional.ifPresent() method performs an action if a value is present. It takes a Consumer as an argument, which is a functional interface that accepts a value and returns no result.
• Get(): Returns the value if present, else throws NoSuchElementException.</td></tr>
<tr><td>Interface Changes</td><td>• Prior to Java 8, interfaces could only declare methods without providing implementations.
• Any class implementing an interface had to provide concrete implementations for all the interface's methods.
• Impact: Adding new methods to an interface would break existing implementations, making it challenging for library creators to evolve interfaces without causing issues.
Java 8 introduced default and static methods in interfaces to address these limitations
<table><tr><td>Default Methods</td><td>Interfaces can now have method implementations using the default keyword. Default methods allow new methods to be added to existing interfaces without breaking existing implementations. If all or most implementing classes will use a similar implementation, use a default method to avoid code</td></tr></table></td></tr>
</table>

<table>
<tr><td></td><td><table>
<tr><td></td><td>duplication also they can be overridden by implementing classes.

Example: The List interface includes a sort() method implemented with the default keyword.</td></tr>
<tr><td>Static Methods</td><td>Interfaces can include static methods. These methods belong to the interface itself and can be called using the interface nam e. Static methods in interfaces allow for utility methods/helper functions to be defined within interfaces, promoting code reuse but not tied to a specific i nstance. Unlike default methods, static methods in interfaces cannot be overridden by implementing classes

Example: Collections.emptyList(); This static method from the Collections class in Java returns an unmodifiable empty list.</td></tr>
<tr><td>Functional Interfaces</td><td>These are interfaces with a single abstract method and serve as the basis for lambda expressions and method references. The @ FunctionalInterface annotation can be used to mark an interface as functional</td></tr>
<tr><td>Method References</td><td>• This feature provides a concise way to refer to existing methods by their names, improving code readability and reuse
• Optional Return Types: The Optional<T> class was introduced to handle nullability more expressively, reducing the risk of null pointer exceptions</td></tr>
</table></td></tr>
<tr><td>Date/Time</td><td>Java 8 introduced a new Date and Time API (java.time package) to address the limitations and design flaws of the old java.util.Date and java.util.Calendar classes

LocalDate today = LocalDate.now();//2024-08-25
LocalDate birthday = LocalDate.of(1990, 8, 25);

LocalTime now = LocalTime.now();//14:30:00
LocalTime meetingTime = LocalTime.of(10, 30);

LocalDateTime now = LocalDateTime.now();//2024-08-25T14:30:00
LocalDateTime event = LocalDateTime.of(2024, 8, 25, 14, 30);</td></tr>
</table>

Stream

26 December 2023 11:50

<table>
<tr><th>Name</th><th>Example</th></tr>
<tr><td>forEach</td><td>Performs an action for each element of the stream.

List<String> strings = Arrays.asList("apple", "banana", "orange");
strings.stream().forEach(System.out::println);</td></tr>
<tr><td>.filter</td><td>Returns a stream consisting of the elements that match the given predicate.

List<String> filteredList = strings.stream().filter(s -> s.startsWith("a")).collect(Collectors.toList());</td></tr>
<tr><td>.map</td><td>Transforms each element using the provided function and replaces it in the stream

List<Integer> lengths = strings.stream().map(String::length).collect(Collectors.toList());</td></tr>
<tr><td>.boxed()</td><td>Converts a primitive stream into a stream of wrapper objects.

Stream<Integer> boxedStream = IntStream.of(numArray).boxed();</td></tr>
<tr><td>mapToInt</td><td>Converts a stream of objects into an IntStream of primitive int values.

IntStream intStreamAgain = Stream.of(1, 2, 3, 4).mapToInt(Integer::intValue);</td></tr>
<tr><td>mapToObj</td><td>mapToObj() is used to convert a primitive stream (like IntStream, DoubleStream, LongStream) to a stream of objects (Stream<U>).

Stream<String> stringStream = IntStream.of(1, 2, 3, 4).mapToObj(Integer::toString); // Converts IntStream to Stream<String></td></tr>
<tr><td>ArrayList to int Array</td><td>arrayList.stream().mapToInt(i->i).toArray()</td></tr>
<tr><td>ArrayList to String Array</td><td>al.toArray(new String[0]);</td></tr>
<tr><td>Integer Array to Array List</td><td>ArrayList<Integer> arrayList = new ArrayList<>(Arrays.asList(intArray));</td></tr>
<tr><td>String array to ArrayList</td><td>ArrayList<String> arrayList = new ArrayList<>(Arrays.asList(strArray));</td></tr>
<tr><td>.flatmap</td><td>Flattens the stream of collections into a single stream.
List<List<String>> nestedList = Arrays.asList(Arrays.asList("apple", "orange"), Arrays.asList("banana"));
List<String> flatList = nestedList.stream().flatMap(Collection::stream).collect(Collectors.toList());</td></tr>
<tr><td>.distinct</td><td>Returns a stream consisting of distinct elements.

List<Integer> distinctNumbers = numbers.stream().distinct().collect(Collectors.toList());</td></tr>
<tr><td>.sorted</td><td>Returns a stream sorted according to the natural order or using a custom comparator.

strings.stream().sorted().collect(Collectors.toList());
strings.stream().sorted(Comparator.reverseOrder()).collect(Collectors.toList());

//Sort with String Length
strings.stream().sorted(Comparator.comparing(x->s.length()).collect(Collectors.toList());</td></tr>
<tr><td>.reduce</td><td>Performs a reduction on the elements of the stream using an associative accumulation function.

<table>
<tr><th>Return Type</th><th>Code</th></tr>
<tr><td>Optional</td><td>Optional<Integer> sum = numbers.stream().reduce(Integer::sum);
Optional<Integer> optProduct=myList.stream().reduce((a,b)->a*b);
System.out.println("Opt Product is "+optProduct.get());</td></tr>
<tr><td>Integer</td><td>int sum=Arrays.stream(array).reduce(0, (a, b) -> a + b);

int sum = Arrays.stream(classPoints).sum();

Int product=Arrays.stream(array).reduce(1, (a, b) -> a * b);

myList.stream().reduce((a, b) -> a > b ? a : b).orElse(Integer.MIN_VALUE);</td></tr>
</table>
</td></tr>
<tr><td>.count</td><td>Returns the count of elements in the stream.

long count = strings.stream().count();</td></tr>
<tr><td>.allMatch</td><td>Checks whether all elements of the stream match the provided predicate.

boolean allEven = numbers.stream().allMatch(n -> n % 2 == 0);</td></tr>
<tr><td>.noneMatch</td><td>Checks whether no elements of the stream match the provided predicate.

boolean noneEven = numbers.stream().noneMatch(n -> n % 2 == 0);</td></tr>
<tr><td>.anyMatch</td><td>Checks whether any elements of the stream match the provided predicate. Returns true if at least one element matches the predicate; otherwise, returns false.

boolean anyEven = numbers.stream().anyMatch(n -> n % 2 == 0);</td></tr>
<tr><td>.findAny</td><td>Returns an Optional containing any element of the stream. We also have findFirst

List<String> names = Arrays.asList("Saurav", "John", "Doe", "Jane", "Smith");

Optional<String> anyName = names.stream().filter(name -> name.startsWith("J")) .findAny();

anyName.ifPresent(System.out::println);</td></tr>
<tr><td>.skip</td><td>Returns a stream with the first n elements skipped.

List<Integer> afterSkip = numbers.stream().skip(2).collect(Collectors.toList());</td></tr>
<tr><td>.limit</td><td>Returns a stream truncated to be no longer than a specified size.</td></tr>
</table>

	List<Integer> limited = numbers.stream().limit(3).collect(Collectors.toList());
.peek	List<String> peekedList = strings.stream().peek(System.out::println).collect(Collectors.toList());
.min/.max	Finds the minimum and maximum element of the stream. Optional<Integer> min = numbers.stream().min(Integer::compareTo); int min = arrayList.stream().min(Integer::compareTo).orElse(Integer.MAX_VALUE); Optional<Integer> max = numbers.stream().max(Integer::compareTo); int min = Arrays.stream(arr).min().getAsInt(); int max = Arrays.stream(arr).max().getAsInt();
Collectors.toMap	Collects the elements of the stream into a Map. List<String> strings = Arrays.asList("apple", "banana", "orange"); Map<Integer, String> lengthToWordMap = strings.stream().collect(Collectors.toMap(String::length, Function.identity()));
.collect()	List<String> strings = Arrays.asList("apple", "banana", "orange"); LinkedList<String> linkedList = strings.stream().collect(Collectors.toCollection(LinkedList::new));// this list in mutable .collect(Collectors.toList()) .collect(Collectors.toSet()) .collect(Collectors.joining()) .collect(Collectors.joining(",")) .collect(counting()) Arrays.stream(phrase.split(" ")) .map(i -> i.substring(0, 1).toUpperCase() + i.substring(1)) .collect(Collectors.joining(" ")); ListToMap List<Integer> list=new ArrayList<>(Arrays.asList(1,2,3)); Map<Integer,Integer> listToMap=list.stream().collect(Collectors.toMap(val->val,val->val*val)); Not used after Java 16: List<String> namesWithA = names.stream().filter(name -> name.startsWith("A")).toList();// this list is Unmodifiable
forEachOrdered	Performs an action for each element of the stream in the encounter order. strings.stream().forEachOrdered(System.out::println);
Switch	Untill java 7 only integers could be used in switch statement. Java 8 added string and enum as case statements Int value=5; Switch(value){ Case 1: sout; Break: Case 2: Default: } Java 12 introduced switch expression return switch (op) { case "+" -> v1 + v2; case "-" -> v1 - v2; case "*" -> v1 * v2; case "/" -> { if (v2 == 0) { throw new ArithmeticException("Division by zero is not allowed."); } yield v1 / v2; } default -> throw new IllegalArgumentException("Invalid operation: " + op); }; Switch expression can have multiple case statements Return switch (day){ Case "monday","Tuesday" ->"weekdays"; };
AtomicInteger	AtomicInteger count = new AtomicInteger() This line initializes an AtomicInteger called count with an initial value of 0. AtomicInteger is used to ensure thread safety when incrementing the counter inside the stream. count.getAndIncrement():retrieves the current value of the counter and then increments it
Stream of Characters	char[] charArray = {'a', 'b', 'c', 'd'}; IntStream charStream = IntStream.range(0, charArray.length).map(i -> charArray[i]); charStream.mapToObj(c -> (char)c).forEach(System.out::println);
Stream of String	Stream.of(s1.concat(s2).split("")).sorted().distinct() .collect(Collectors.joining()); var chars = str.split(""); return IntStream.range(0, chars.length).mapToObj(i -> chars[i].toUpperCase() + chars[i].toLowerCase().repeat(i)) .collect(Collectors.joining("-")); String input = "sumitM28"; input.chars().filter(Character::isLowerCase).forEach(System.out::print); Chars() returns a stream of int
Stream of Array	int[] arr = {1, 23, 123, 45, 134};

	Arrays.stream(arr).filter(num -> String.valueOf(num).startsWith("1")).forEach(System.out::println);
Stream of Map	Map<String, Integer> filteredMap = sampleMap.entrySet() .stream() .filter(entry -> entry.getValue() > 20) // Filter entries with value > 20 .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
Print HashMap	map.forEach((key, value) -> System.out.println(key + " -> " + value)); for (Map.Entry<Integer, Integer> entry : countMap.entrySet()) { if (entry.getValue() % 2 != 0) { return entry.getKey(); } }
Intstream	• IntStream operates on primitive int values directly rather than boxed Integer objects. This reduces memory overhead because it avoids the cost of autoboxing (converting primitives to their wrapper types). • IntStream can be parallelized easily using the .parallel() method. • IntStream provides specialized reduction operations that are more efficient than their generic counterparts. Examples include sum(), average(), min(), and max(), which operate directly on int values without the overhead of boxing. • IntStream includes methods that are tailored specifically for dealing with int primitives, like range() and rangeClosed(), which generate streams of numbers over specified ranges. • Intstream.range(1,50)->1-49 • Intstream.rangeClosed(1,50)->1-50
Parallel Sream	When to Use Parallel Stream • **Large datasets**: Parallelism shines with more elements to process. • **CPU-intensive operations**: Tasks that require computation benefit from multi-core processing. • **Independent operations**: Each task should be self-contained without shared mutable state. When Not to Use Parallel Stream • **Small datasets**: Thread overhead may result in slower execution. • **I/O-bound operations**: These don't benefit from CPU parallelism and may block threads. • **Stateful operations**: Operations like sorting that rely on shared or mutable state can yield incorrect or unpredictable results.

Solid Principles

18 February 2024 08:36

SRP

Every software component(Class, method or module) should have only one Responsibility(Ony 1 Reason to Change).

- **Cohesion**: Degree to which various components of software are related. Bifurcate closely related method to increase cohesion.
- **Coupling**: Level of interdependency between software Components. Aim for High Cohesion in a component and loose coupling.

Low Cohesion

```
public class Square {

    int side = 5;

    public int calculateArea() {
        return side * side;
    }

    public int calculatePerimeter() {
        return side * 4;
    }

    public void draw() {
        if (highResolutionMonitor) {
            // Render a high resolution image of a square
        } else {
            // Render a normal image of a square
        }
    }

    public void rotate(int degree) {
        // Rotate the image of the square clockwise to
        // the required degree and re-render
    }

}
```

High Cohesion

```
public class Square {

    int side = 5;

    public int calculateArea() {
        return side * side;
    }

    public int calculatePerimeter() {
        return side * 4;
    }
}
```

```
public class SquareUI {

    public void draw() {
        if (highResolutionMonitor) {
            // Render a high resolution image of a square
        } else {
            // Render a normal image of a square
        }
    }

    public void rotate(int degree) {
        // Rotate the image of the square clockwise to
        // the required degree and re-render
    }
}
```

Tightly Coupled

```
public class Student {

    private String studentId;
    private Date studentDOB;
    private String address;

    public void save() {
        // Serialize object into a string representation
        String objectStr = MyUtils.serializeIntoAString(this);
        Connection connection = null;
        Statement stmt = null;
        try {
        Class.forName("com.mysql.jdbc.Driver");
        connection =
        DriverManager.getConnection("jdbc:mysql://localhost:3306/
        MyDB", "root", "password");
        stmt = connection.createStatement();
        stmt.execute("INSERT INTO STUDENT VALUES (" + objectStr +
        ")");
        } catch (Exception e) {
                    e.printStackTrace();
        }
    }

    public String getStudentId() {
        return studentId;
    }

    public void setStudentId(String studentId) {
        this.studentId = studentId;
    }

    ...
    ...
    ....
    ......
}
```

The Student class may have multiple reasons to change:
- Change in student ID format
- Change in student name format
- Change in database backend

Loosly Coupled

```
public class Student {

    private String studentId;
    private Date studentDOB;
    private String address;

    public void save() {
        new StudentRepository().save(this);
    }

    public String getStudentId() {
        return studentId;
    }

    public void setStudentId(String studentId) {
        this.studentId = studentId;
    }

    ...
    ...
    ....
    ......
}
```

```
public class StudentRepository {
    public void save(Student student) {
        // Serialize object into a string representation
        String objectStr = MyUtils.serializeIntoAString(student);
        Connection connection = null;
        Statement stmt = null;
        try {
            Class.forName("com.mysql.jdbc.Driver");
            connection =
            DriverManager.getConnection("jdbc:mysql://localhost:
            3306/MyDB", "root", "password");
            stmt = connection.createStatement();
            stmt.execute("INSERT INTO STUDENT VALUES (" +
            objectStr + ")");
        } catch (Exception e) {
                e.printStackTrace();
        }
    }
}
```

Student Class	Student Repository
- Change in student ID format - Change in student name format	- Change in database backend

OCP

S/W Components should be closed for modification but open for extention. Playstation is closed for modification but can be extended by PS VR, headsets.

- Classes should allow for extension through inheritance or composition.
- Existing code should not be modified when extending functionality.
- Promotes code reuse and minimizes the risk of introducing bugs in existing code.
- Example: Child class overiding parent class function

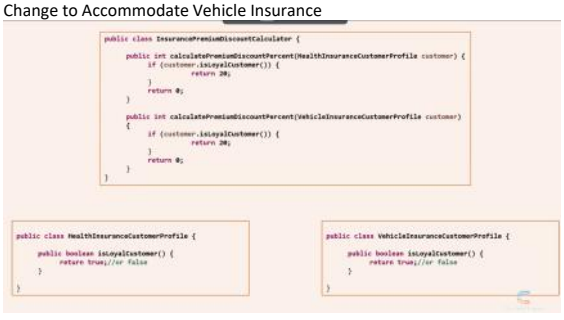
Open for Extension, Closed for Modification: i.e You should put new code in New Classes/Modules or Reuse existing code via Inheritance. Existing code should be modified only for Bug fixing.

If we want to add Vehicle Insurance product to our portfolio then InsurancePremiumDiscountCalculator class needs to be modified violating OCP principle.

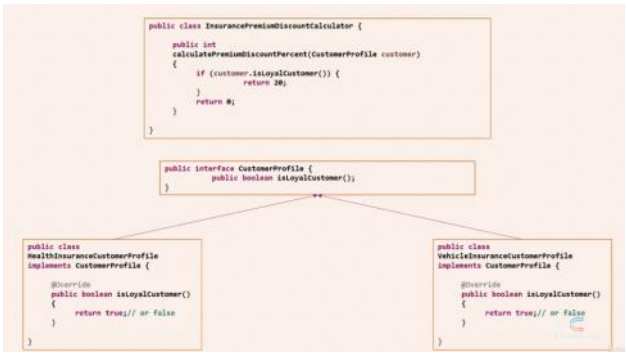
```
public class InsurancePremiumDiscountCalculator {

    public int
    calculatePremiumDiscountPercent(HealthInsuranceCustomerProfile customer)
    {
        if (customer.isLoyalCustomer()) {
            return 20;
        }
        return 0;
    }
}
```

```
public class HealthInsuranceCustomerProfile {

    public boolean isLoyalCustomer() {
        return true;//or false
    }

}
```

Change to Accommodate Vehicle Insurance



OCP Design




```
// Without Open-Closed Principle
class Operation{
    double calculate(doulbe a1, double a2, string operationType){
        switch(operationType){
            case "+":
                return a1 + a2;
            case "-":
                return a1 - a2;
            default:
                // Throw some exceptions
        }
        return 0;
    }
}
```

```
// With Open-Closed Principle
interface Operation{
     double calculate(double a1, double a2);
}

class AddOperation implements Operation{
    double calculate(double a1, double a2){
         return a1 + a2;
    }
}

class DivisionOperation implements Operation{
    double calculate(double a1, double a2){
         return a1/a2;
    }
}
```

Selenium Example: Factory Design Pattern

```
public interface DriverManager {
  WebDriver getDriver();
}

public class ChromeDriverManager implements DriverManager {
  public WebDriver getDriver() {
    return new ChromeDriver();
  }
}

public class FirefoxDriverManager implements DriverManager {
  public WebDriver getDriver() {
    return new FirefoxDriver();
  }
}

// Add new browser support without modifying existing classes
public class EdgeDriverManager implements DriverManager {
  public WebDriver getDriver() {
    return new EdgeDriver();
  }
}

public class WebDriverFactory {
  public static DriverManager getManager(String browserType) {
    switch (browserType) {
      case "chrome":
        return new ChromeDriverManager();
      case "firefox":
        return new FirefoxDriverManager();
      case "edge":
        return new EdgeDriverManager(); // New functionality added
      default:
        throw new IllegalArgumentException("Invalid browser type");
    }
  }
}
```

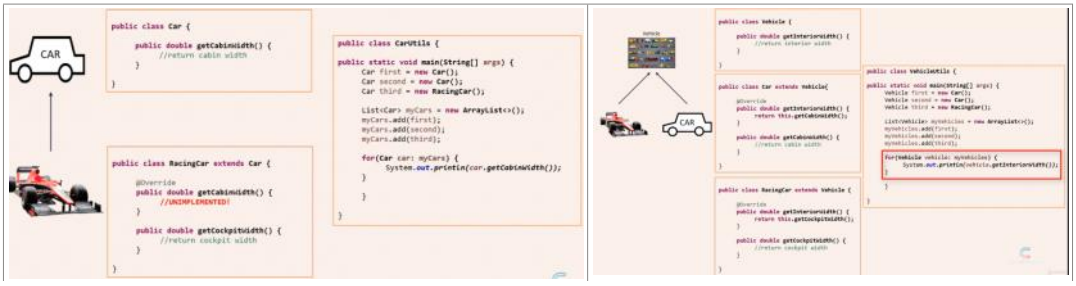
LSP

Objects of a superclass should be replaceable with objects of a subclass without affecting the correctness of the program.

- Subtypes must be substitutable for their base types.
- The behavior of a subclass should not contradict the behavior of its superclass.
- In order to obey this law, the Child class should support all methods of Parents class.

Example: Ostrich **is-a** bird but is cannot fly so we cannot substitute ostrich's fly method with bird's fly method.

RacingCar **is-a** Car(inheritance) but getCabinWidth() of Car class cannot be called on RacingCar Object.



Selenium provides the WebDriver interface with multiple implementations like ChromeDriver, FirefoxDriver, EdgeDriver, etc. These implementations adhere to the LSP because they can replace the WebDriver reference without breaking functionality.

ISP

The Interface Segregation Principle (ISP) states that no client should be forced to depend on methods it does not use.

In a real-world scenario, this principle can be illustrated with office equipment like printers, scanners, and fax machines. For instance, if we create an interface named IMultiFunction to represent a multi-function device, it may have methods for printing, scanning, and faxing. However, if we implement this interface for devices that cannot perform all these functions, we end up with blank method implementations, which violates the ISP.

This violation becomes problematic when other parts of the code, such as an employee portal application, rely on these interfaces. Developers may mistakenly assume that all methods are fully implemented, leading to potential errors and breaking of the code.

```
public interface IMultiFunction {
	public void print();
	public void getPrintSpoolDetails();
	public void scan();
	public void scanPhoto();
	public void fax();
	public void internetFax();
}
```

Selenium uses multiple interfaces to segregate responsibilities, allowing a class to implement only what it needs. For example:

- SearchContext: Defines methods for finding elements (findElement and findElements).
- WebDriver: Extends SearchContext and adds browser-related operations like get, navigate, and manage.
- TakesScreenshot: Used by drivers that support screenshots.
- JavascriptExecutor: Used by drivers that support JavaScript execution.

Not all drivers need to implement every interface.

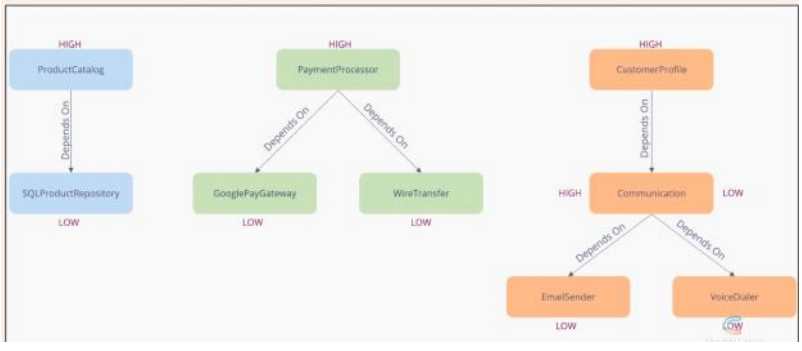
DIP

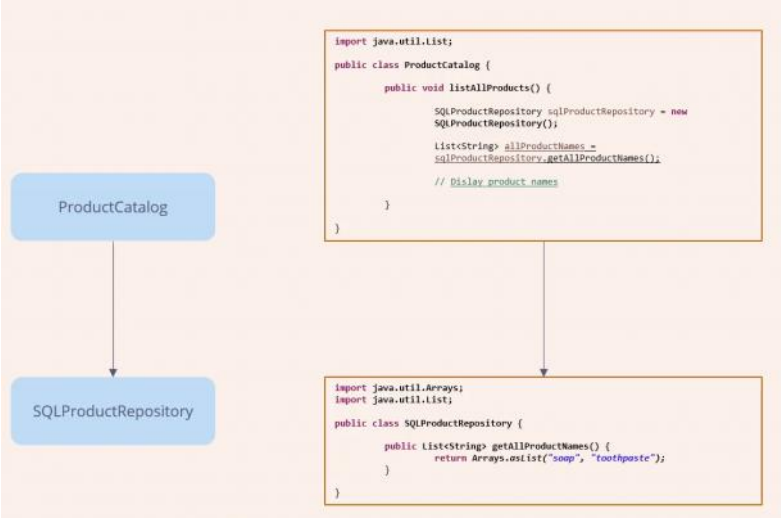
The Dependency Inversion Principle emphasizes that our class should depend upon interfaces or abstract classes instead of Concrete class and functions.

Example: we use Webdriver driver =new chromedriver and not the other way around.

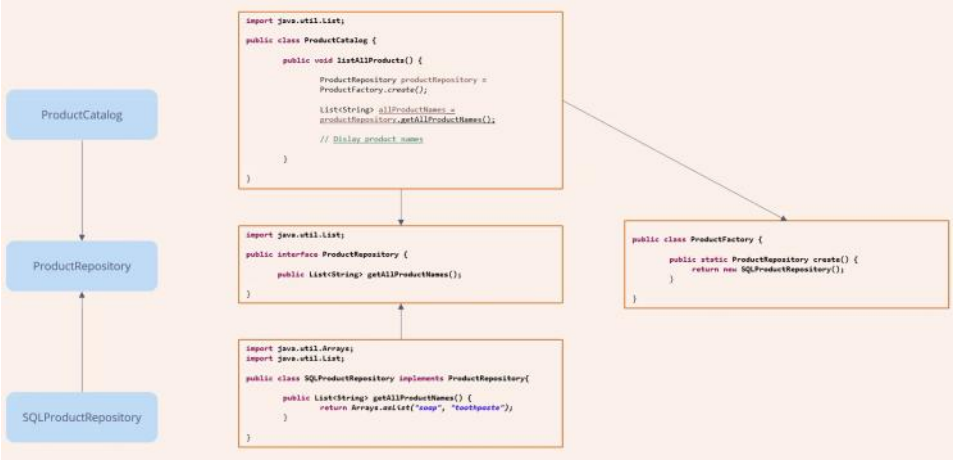
In a scenario where an eCommerce application comprises high-level modules like ProductCatalog, PaymentProcessor, and CustomerProfile, and low-level modules like SQLProductRepository and GooglePayGateway, dependencies must be carefully managed.

Initially, if high-level modules directly depend on low-level modules, it violates the DIP. For example, if ProductCatalog directly depends on SQLProductRepository, it creates tight coupling and violates the principle.





To address this violation, abstractions like interfaces are introduced. In the example, a ProductRepository interface is created, and SQLProductRepository implements it. A factory method is used to instantiate the concrete implementation, removing direct dependencies between high-level and low-level modules.



After restructuring, both high-level and low-level modules depend on abstractions, adhering to the DIP. Additionally, the principle requires that abstractions should not depend on details, leading to an inversion of dependency where details depend on abstractions.

File Handeling

07 July 2026 16:10

<table>
<tr><td>File I/O</td><td>File-> FileInputStream-> InputStreamReader-> BufferedReader-> User Program</td></tr>
<tr><td>Data Types</td><td>• Binary(Bytes)
• InputStream
• OutputStream
• Text
• Reader
• Writer</td></tr>
<tr><td>Byte Streams</td><td>This is used to process data byte by byte (8 bits) useful for files. Though it has many classes, the FileInputStream and the FileOutputStream are the most popular ones. The FileInputStream is used to read from the source and FileOutputStream is used to write to the destination.
<table>
<tr><th>Stream class</th><th>Description</th></tr>
<tr><td>BufferedInputStream</td><td>It is used for Buffered Input Stream.</td></tr>
<tr><td>DataInputStream</td><td>It contains method for reading java standard datatypes.</td></tr>
<tr><td>FileInputStream</td><td>This is used to reads from a file</td></tr>
<tr><td>InputStream</td><td>This is an abstract class that describes stream input.</td></tr>
<tr><td>PrintStream</td><td>This contains the most used print() and println() method</td></tr>
<tr><td>BufferedOutputStream</td><td>This is used for Buffered Output Stream.</td></tr>
<tr><td>DataOutputStream</td><td>This contains method for writing java standard data types.</td></tr>
<tr><td>FileOutputStream</td><td>This is used to write to a file.</td></tr>
<tr><td>OutputStream</td><td>This is an abstract class that describe stream output.</td></tr>
</table></td></tr>
<tr><td>FileInputStream</td><td>This reads Bytes. Used for images, pdf but not for text

FileInputStream fis = new FileInputStream("image.png");

int data;

while((data = fis.read()) != -1){
System.out.println(data);
}

returns byte value like
137
80
78
71
…</td></tr>
<tr><td>FileOutputStream</td><td>Write bytes.

FileOutputStream fos =new FileOutputStream("image.png");

fos.write(byteArray);

Advantages
• Fast
• Simple
• Binary safe
Disadvantages
• Cannot understand text
• Reads byte by byte unless buffered</td></tr>
<tr><td>Character Streams</td><td>In Java, characters are stored using Unicode conventions. Character stream automatically allows us to read/write data character by character. Though it has many classes, the FileReader and the FileWriter are the most popular ones. FileReader and FileWriter are character streams used to read from the source and write to the destination respectively.

<table>
<tr><th>Stream class</th><th>Description</th></tr>
<tr><td>BufferedReader</td><td>It is used to handle buffered input stream.</td></tr>
<tr><td>FileReader</td><td>This is an input stream that reads from file.</td></tr>
<tr><td>InputStreamReader</td><td>This input stream is used to translate byte to character.</td></tr>
<tr><td>OutputStreamReader</td><td>This output stream is used to translate character to byte.</td></tr>
<tr><td>Reader</td><td>This is an abstract class that define character stream input.</td></tr>
<tr><td>PrintWriter</td><td>This contains the most used print() and println() method</td></tr>
<tr><td>Writer</td><td>This is an abstract class that define character stream output.</td></tr>
<tr><td>BufferedWriter</td><td>This is used to handle buffered output stream.</td></tr>
<tr><td>FileWriter</td><td>This is used to output stream that writes to file.</td></tr>
</table></td></tr>
<tr><td>FileReader</td><td>Reads text.

FileReader reader = new FileReader("notes.txt");

Now Java converts bytes into characters.</td></tr>
<tr><td>FileWriter</td><td>OutputStreamWriter(new FileOutputStream(...),StandardCharsets.UTF_8)</td></tr>
<tr><td>Buffered Streams</td><td>FIS reads 1 Byte at a time, this makes large files reading a very intensive operation.</td></tr>
</table>

	BufferedReader: Read 8 KB, Keep in memory and serve from memory.
BufferedReader	Efficient text reading. BufferedReader reader =new BufferedReader(new FileReader(file)); Advantages • Fast • readLine() • Memory efficient • Reads chunk by chunk Disadvantages • Still low level. • Need loop.
BufferedInputStream	Exactly same idea except It buffers bytes instead of characters.
BufferedOutputStream	Buffers byte writing. Very useful when copying files. BufferedOutputStream bos =new BufferedOutputStream(new FileOutputStream(...));
Files.lines()	public static void readWithStream(Path path) { try (Stream<String> lines = Files.lines(path)) { lines.forEach(System.out::println); } catch (IOException e) { System.err.println("Error reading from file: " + e.getMessage()); e.printStackTrace(); } }
Files.writeString	public static void writeWithString(Path path) { try { Files.writeString(path, "Hello World\n" + Math.random() + "\n", StandardOpenOption.CREATE, StandardOpenOption.APPEND); } catch (IOException e) { System.err.println("Error writing to file: " + e.getMessage()); e.printStackTrace(); } }
readWriteWithBuffer	private static void readWriteWithBuffer() { Path path = Paths.get("src/test/resources/file3.txt"); try (BufferedWriter writer = Files.newBufferedWriter(path, StandardOpenOption.CREATE, StandardOpenOption.APPEND)) { writer.write("Hello World"); writer.newLine(); writer.write(String.valueOf(Math.random())); writer.newLine(); } catch (IOException e) { // Caught specific IOException instead of generic Exception System.err.println("Error writing to file: " + e.getMessage()); e.printStackTrace(); } try (BufferedReader reader = Files.newBufferedReader(path)) { String line; while ((line = reader.readLine()) != null) { System.out.println(line); } } catch (IOException e) { System.err.println("Error reading from file: " + e.getMessage()); e.printStackTrace(); } }
readWriteWithStream	private static void readWriteWithStream() { Path path = Paths.get("src/test/resources/file3.txt"); try { Files.writeString(path, "Hello World\n" + Math.random() + "\n", StandardOpenOption.CREATE, StandardOpenOption.APPEND); } catch (IOException e) { System.err.println("Error writing to file: " + e.getMessage()); e.printStackTrace(); } try (Stream<String> lines = Files.lines(path)) { lines.forEach(System.out::println); } catch (IOException e) { System.err.println("Error reading from file: " + e.getMessage()); e.printStackTrace(); } }
Line By Line Compare	public static void compareFiles() { Path path1 = Paths.get("src/test/resources/file1.txt"); Path path2 = Paths.get("src/test/resources/file2.txt"); try (Stream<String> s1 = Files.lines(path1); Stream<String> s2 = Files.lines(path2)) {

```
        Iterator<String> it1 = s1.iterator();
        Iterator<String> it2 = s2.iterator();
        int lineNum = 1;

        while (it1.hasNext() || it2.hasNext()) {
            String line1 = it1.hasNext() ? it1.next() : "<no line>";
            String line2 = it2.hasNext() ? it2.next() : "<no line>";

            if (!line1.equals(line2)) {
                System.out.printf("Difference at line %d:\nFile1: %s\nFile2: %s\n\n", lineNum, line1, line2);
            }
            lineNum++;
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

Comparison

Feature	Files.readString()	Files.readAllLines()	Files.lines()
Return Type	String	List<String>	Stream<String>
Memory Footprint	**High** (Entire file in RAM)	**High** (Entire file in RAM)	**Low** (Constant, handles line-by-line)
File Size Limit	Small files only	Small files only	**Unlimited** (Safe for GB+ files)
Try-with-resources?	No	No	**Yes** (Mandatory to avoid leaks)
Java Version	Java 11+	Java 7+	Java 8+
	Use this when your framework needs to quickly read/write small runtime configurations, environment properties, or small JSON payloads.		Essential for parsing heavy automation execution logs. It streams line-by-line using a Java Stream, keeping memory consumption near zero even if the log file is 10 GB.

File Truncation

- StandardOpenOption.CREATE: Creates the file if it doesn't exist.
- StandardOpenOption.TRUNCATE_EXISTING: If the file exists, it cuts its length to 0 (wipes old test logs from previous runs).
- StandardOpenOption.APPEND: Keeps old data and appends new test steps to the bottom.

Generics

18 July 2026 08:59

Generics	Generics in Java refer to parameterized types that allow writing code which works with multiple data types using a single cla ss, interface, or method. They improve reusability and ensure type safety at compile time. Standard Naming Conventions: • T — Type • E — Element (used heavily in Collections) • K — Key • V — Value • N — Number
Generic Class	`public class GenericPair<T, U> {` `  private T first;` `  private U second;` `  public GenericPair(T first, U second) {` `  this.first = first;` `  this.second = second;` `  }` `  public T getFirst() { return first; }` `  public void setFirst(T first) { this.first = first; }` `  public U getSecond() { return second; }` `  public void setSecond(U second) { this.second = second; }` `}`
Generic Methods	`public class MethodUtility {` `  // Generic method to print any array type` `  public static <T> void printArray(T[] array) {` `  for (T element : array) {` `  System.out.print(element + " ");` `  }` `  System.out.println();` `  }` `  public static void main(String[] args) {` `  Integer[] intArray = {1, 2, 3, 4, 5};` `  String[] stringArray = {"Hello", "World"};` `  // Type is inferred automatically from arguments` `  printArray(intArray);` `  printArray(stringArray);` `  }` `}`
Covariance	Covariance allows a subtype reference to be assigned to a supertype reference. Arrays are covariant: String[] is a subtype of Object[]. `Number[] numArray = {1, 2, 3};` `Object[] objArray = numArray; // Valid compile-time syntax` `objArray[0] = "Hello";  // Throws ArrayStoreException at runtime!` **Collections are NOT covariant:** To prevent the runtime crash shown above, List<String> is **not** a subtype of List<Object>. List<Employee> list = new ArrayList<Developer>(); // Compilation error! To resolve this limitation while preserving type safety, Java uses **Wildcards (?)**.
Upper Bounded Wildcards (<? extends T>)	Restricts the unknown type to be either type T or a child class of T. `// Works for List<Employee>, List<Developer>, List<Manager>` `public static void printEmployeeNames(List<? extends Employee> employees) {` `  for (Employee e : employees) {` `  System.out.println(e.getName()); // Safe to read as an Employee` `  }` `  // employees.add(new Developer()); // COMPILATION ERROR!` `  // Reason: The compiler doesn't know if the list is actually a List<Manager>.` `}` **Rule:** Use <? extends T> when you only need to **read** data out of a structure. You cannot add elements to an upper-bounded collection
Lower Bounded Wildcards (<? super T>)	Restricts the unknown type to be either type T or a parent class of T. `// Works for List<Integer>, List<Number>, List<Object>` `public static void addNumbers(List<? super Integer> list) {` `  for (int i = 1; i <= 10; i++) {` `  list.add(i); // Safe to write/add an Integer` `  }` `}` **Rule:** Use <? super T> when you need to **write/add** data into a structure.
Unbounded Wildcards (<?>)	Represents an entirely unknown type. It is useful when your method only relies on functionality provided by the base Object c lass or methods that don't care about the type parameter (like List.size() or List.clear()). // List<Object> cannot accept a List<Integer>. List<?> can accept any type. `public static void printList(List<?> list) {` `  for (Object elem : list) {` `  System.out.print(elem + " ");` `  }`

```
    System.out.println();
}
```

Miscellaneous

13 November 2024 09:22

<table>
<tr><td>Points to Remember</td><td>
• The java.lang.System.exit() method terminates the Java virtual machine, accepting a status code. Non -zero values typically indicate abnormal termination.
• System.out.print: System is a final class of java.lang, out is a static variable of type printstream, and print is a method of printstream cl ass.
• Java automatically converts long to int if 'L' is not used in initialization.
• When a package is imported, its sub-packages aren’t imported; import them separately if needed.
• Main method can be overloaded, and another class's main method can be called in a main method. Main method cannot be overridd en
• When creating an object of a subclass, the parent class constructor is executed first, then the child class constructor. Comp iler adds super() as the first statement in the child class constructor.
• Default constructor cannot be called if a parameterized constructor is present unless explicitly declared.
• Length is a property of array while a function in string
• Integer int1=new Integer(5) will create new objects and == comparison won't work as in Strings. So create Integer variables w ith Integer.valueOf(5)
• main() is static for JVM to call it without creating an instance.
• throw: Explicitly throws an exception.
• throws: Specifies checked exceptions thrown by a method.
• Throwable: Superclass of all errors and exceptions.
• For primitive variable we store its value in stack
• Reference Data Types (or Non-Primitive) do not store the actual value directly in the variable. Instead, they store a memory address (reference) pointing to where the object is located in the Heap Memory.We cannot overload methods that differ only by static keyword
• \n for new line
• Static blocks are executed first, followed by the main method. These are used to initialize static variables, loading native libs or doing complex one time setup i.e DB Connections.
• Static block is executed only once: the first time the class is loaded into memory. A class can have multiple static blocks, and they are executed in the order they appear in the source code. It can be used to initilise static variables. Created and destroyed with the program.
• Instance variable are stored in heap and automatically initialized.
• Static block Main function's string[] can accept command line arguments.
• True, false and null are literals(value after = symbol like int num=8, 8 is a literal) not keywords.
• Variables:
  • Must start with a letter, $, or _. Cannot start with a digit.
  • int myVar and int MyVar are different variables.
  • Cannot use Java keywords (e.g., new, class, for, int).
  • Use lowercase for single words (city). Use camelCase for multiple words (jaipurCity).
  • Use ALL_CAPS for constant/final variables.
• Factory Method: A static method that returns an instance of a class.
</td></tr>
<tr><td>Array Declaration</td><td>
<pre>
int[] a =new int[5];
a[0]=5;
System.out.println(a[0]);

int[] b ={1,2,3,4,5};
System.out.println(b[2]);

int[] b = new int[]{1, 2, 3, 4, 5};//Anonymous Array, can be used anywhere like printNumbers(new int[]{10, 20, 30});

int[][] c =new int[2][2];
c[0][0]=2;
System.out.println(c[0][0]);

int[][] d = {{1,2,3},{5,6,7}};
System.out.println(d[1][0]);
</pre>
</td></tr>
<tr><td>System.out.printf</td><td>
<table>
<tr><th>Specifier</th><th>Description</th></tr>
<tr><td>%d</td><td>Decimal integer (int, long)</td></tr>
<tr><td>%f</td><td>Floating-point number (float, double)</td></tr>
<tr><td>%s</td><td>String</td></tr>
<tr><td>%c</td><td>Character</td></tr>
<tr><td>%b</td><td>Boolean</td></tr>
<tr><td>%x</td><td>Hexadecimal integer</td></tr>
<tr><td>%o</td><td>Octal integer</td></tr>
<tr><td>%n</td><td>newline character (equivalent to System.lineSeparator())</td></tr>
</table>
• int i = 42; double pi = 3.14159; String name = "Saurav";
• System.out.printf("The number is: %d%n", i);//42;
  ○ Integers: %d is for decimals (integers), %n is a newline
• System.out.printf("The value of pi is: %.2f%n", pi);//3.14
  ○ Floating point: %.2f rounds to 2 decimal places
• String name = "Saurav"; System.out.printf("Hello, %s!%n", name);//Hello, Saurav!
• System.out.printf("Pi with width 3 and precision 2: %3.2f%n", pi);//3.14
  ○ Width and Precision: %3.2f
  ○ Width 3 means at least 3 characters total; .2 means 2 after the decimal
</td></tr>
<tr><td>Accepting user input</td><td>
<table>
<tr><td>Scanner</td><td>
• Fixed Buffer Size: 1KB
• Slow Performance
• Not thread-safe
• Versatile; automatically parses primitives (nextInt(), etc.).
• <code>Scanner scanner = new Scanner(System.in);</code>
• Int: int intValue = scanner.nextInt();
• Double: double doubleValue = scanner.nextDouble();
• String: String stringValue = scanner.nextLine();
• Boolean: boolean booleanValue = scanner.nextBoolean();
• Char: char charValue = scanner.next().charAt(0);
• If you call scanner.nextInt() followed immediately by scanner.nextLine(), the nextLine() will read the leftover newline character (\n) from when you hit Enter, resulting in an empty string. Add a Dummy nextLine() Call to solve this.
• hasNext(), hasNextLine(), hasNextInt(), hasNextFloat(), hasNextDouble(), etc. (Returns true if the next token matches the specified type).
<pre>
// Reading from Console
</pre>
</td></tr>
</table>
</td></tr>
</table>

<table>
<tr><td></td><td>
<table>
<tr><td></td><td>

```
Scanner sc = new Scanner(System.in);
if(sc.hasNextInt()) { int val = sc.nextInt(); }
sc.close();

// Reading from File
Scanner scFile = new Scanner(new File("file.txt"));
while (scFile.hasNextLine()) { System.out.println(scFile.nextLine()); }
scFile.close();
```

</td></tr>
<tr><td>BufferedReader</td><td>• Default Buffer size 8KB
• Faster performance
• Thread safe
• Reads only strings/chars; requires manual parsing (e.g., Integer.parseInt()).

```
BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));

System.out.print("Enter your name: ");
String name = reader.readLine(); // Reads a whole line

System.out.print("Enter your age: ");
int age = Integer.parseInt(reader.readLine()); // Must manually parse to int
Reader.close();
```

</td></tr>
</table>
</td></tr>
<tr><td>Java.lang package</td><td>The java.lang package is a core part of the Java programming language. It's automatically imported by the Java compiler, so you don’t need to import it explicitly. This package contains fundamental classes that are essential for the Java language to function.

Here are a few key classes:
• Object: The root class of the Java class hierarchy. Every class has Object as a superclass.
• String: Represents a sequence of characters. Immutable and widely used for text manipulation.
• Math: Provides methods for performing basic numeric operations like exponentiation, logarithms, and trigonometric functions.
• System: Contains several useful class fields and methods. It provides access to system resources such as standard input, output, and error streams.
• Thread: Represents a thread of execution in a program. Used for concurrent programming</td></tr>
<tr><td>java.lang.Object</td><td>The Object class belongs to the java.lang package in the java.base module and serves as the root ancestor for every class in Java. It sits at the top of the class hierarchy and has no parent class itself.

• Polymorphic Behavior: Because Object is the universal superclass, a reference variable of type Object can hold a reference to an instance of any class.
• Downcasting: You cannot directly assign an Object reference to a specific subtype variable without explicit casting. Attempting to do so causes a compilation error. If the runtime type is incompatible during a cast, a ClassCastException is thrown. Use instanceof to safely verify the type before casting.

```
Object obj = new Person(); // Valid upcasting
Person p = (Person) obj;    // Valid downcasting
// Safe checking with instanceof
if (obj instanceof Person) {
    Person person = (Person) obj;
}
```

<table>
<tr><td>Final Methods</td><td>• getClass(): A public final native Class<?> getClass() method that returns the runtime class representation of the object. The native keyword indicates the implementation is written in platform-dependent languages like C/C++ and managed by the JVM.
• notify(): Wakes up a single thread waiting on the object's monitor.
• notifyAll(): Wakes up all threads waiting on the object's monitor.
• wait(): Overloaded methods that place a thread in the object's wait queue with or without a timeout.</td></tr>
<tr><td>Non-Final Methods</td><td>• toString(): Returns a string representation of the object. The default format is <fully-qualified-class-name>@<hash-code-in-hexadecimal>. It is standard practice to override this to present clean attribute states while avoiding sensitive data exposure.
• equals(Object obj): Compares objects for similarity. The default behavior uses the == reference operator, checking if both variables point to the same memory location. Classes commonly override it to compare attribute values instead.
• hashCode(): Returns an int value mapping the object's state. The default logic converts the object's memory address into an integer. It optimizes lookups in hash-based collections like HashMap and HashSet.
• clone(): Protected native method used to duplicate an object. Requires the class to implement the Cloneable marker interface, otherwise it throws a CloneNotSupportedException.
• finalize(): Invoked by the garbage collector before an object is destroyed. It is highly unpredictable and has been deprecated since Java 9 in favor of try-with-resources or try-finally structures.</td></tr>
<tr><td>Contract Between equals() and hashCode()</td><td>When overriding these methods, you must adhere to the following strict guidelines to prevent malfunctioning hash collections:
1. If a.equals(b) is true, then a.hashCode() == b.hashCode() must be true.
2. If a.hashCode() == b.hashCode(), a.equals(b) is NOT guaranteed to be true (Hash collisions).
3. Consistency: hashCode() must return the exact same integer value throughout multiple executions if no object data changes.
4. Shared Fields: You must compute hashCode() using the exact same instance variables used inside the equals() logic.
5. Utility Helper: Java 7 introduced java.util.Objects.hash(...) as the recommended approach to calculate hash codes across multiple attributes.
Formal Guidelines for equals()
• Reflexivity: a.equals(a) must always return true.
• Symmetry: If a.equals(b) is true, b.equals(a) must be true.
• Transitivity: If a.equals(b) is true and b.equals(c) is true, then a.equals(c) must be true.
• Consistency: Multiple invocations yield the identical output if internal states remain unchanged.
• Null Handling: Must safely return false if compared with null to avoid exceptions.</td></tr>
</table>
</td></tr>
<tr><td>Record Class</td><td>Records offer a clean, boilerplate-free alternative for declaring immutable data carrier classes.


```
public record Person(String name, String occupation) { }
```

The parameters inside the parentheses are the record components declared in the record header. Under the hood, a record automatically provides:
• A Canonical Constructor matching all header components.
• Getter methods named after the fields directly (e.g., person.name() instead of getName()).
• Default, data-validated versions of equals(), hashCode(), and toString().

Constructors in Records
You can explicitly write out the full canonical constructor, or leverage a Compact Constructor which lacks parameter lists or manual field assignments entirely:
Java

```
public record Person(String name, String occupation) {
    // Compact constructor form
```

</td></tr>
</table>

	public Person { if (name == null || occupation == null) { throw new IllegalArgumentException("Fields cannot be null"); } // Field assignments happen automatically at the end! } } **Rules & Restrictions:** • Records are implicitly final and cannot extend other classes because they already inherit from java.lang.Record. They can, however, implement interfaces. • You cannot add non-static instance fields or instance initializers; you can only add static variables, static methods, or instance methods. • java.lang.Class features two reflection helpers: isRecord() and getRecordComponents()
Var	**Local Variable Type Inference (var) (Java 10+)** The var identifier allows developers to omit explicit type signatures for local variables, instructing the compiler to infer the exact type from the right-hand initialization expression. Java var message = "Hello, World!"; // Infers String var list = new ArrayList<String>(); // Minimizes redundancy **Key Differences & Mechanics:** • var is a **reserved identifier**, not a keyword; it will not conflict with older code using it as a variable or method name. • Java remains strongly typed. Once inferred, the type is fixed and cannot accept incompatible objects later. • Unlike JavaScript where variables lack structural type bounds, Java's var resolves entirely at compile-time. **Strict Constraints of var:** 1. **Must have an immediate assignment**; you cannot write var name; or assign it to null directly. 2. **Cannot** be used for class field declarations. 3. **Cannot** be used as method parameters or method return types. 4. In inheritance scenarios, var infers the exact initializer type rather than the parent class type. // Inheritance type demonstration var s = new Student(); // Type is inferred as Student, not Person s = new Artist(); // Compilation Error: Incompatible types
Variable Arguments in Java	- Variable arguments, or varargs, allow a method to accept a variable number of parameters. - Introduced in Java 5, varargs simplify method invocation when the number of arguments is not known at compile time. Declaration Syntax: - returnType methodName(dataType... parameterName) - The ellipsis (...) denotes variable arguments and must be placed after the parameter type. - The variable arguments parameter acts like an array within the method. // Valid: Varargs is last void sum(String label, int... numbers) { int total = 0; for (int n : numbers) total += n; System.out.println(label + total); } Sum(1,2,3,4,);
Initializers	• **Instance Initialization Block (IIB):** A block of code inside braces { ... } placed outside methods. It executes **every time** an instance is created(constructor is called), running right before the constructor. Useful for sharing initialization logic across overload ed constructors or initializing anonymous classes. Allows you to use **conditional logic** (if-else), **loops**, or **error handling** (try-catch blocks) which cannot be done inline(Assigning values directly while declaring variable). Can use this keyword. public class Car { String model; String color; int horsePower; { this.model = "Camry"; this.color = "Black"; this.horsePower = 200; } } • **Static Initialization Block (SIB):** Declared using the static { ... } block. It executes **exactly once** when the class definition is loaded into memory by the JVM, executing even before the main() method runs. Executed before Main and Constructors. Program with only static block will not run. The java command loads the definition of a given class before it tries to execute its main() method. When the definition of t he class is loaded into memory, at that time the class is initialized, and its static initializer will be executed. This is the reason that you see the code from the static initializer executed before you see the code from the main() method executes. public class InventoryConfig { // Static variables belong to the class, not any single object public static String warehouseLocation; public static List<String> supportedCategories = new ArrayList<>(); // Static Initializer Block static { System.out.println("Static block executed! Loading configurations..."); // 1. Complex logic to set a simple variable String env = System.getenv("APP_ENV"); if ("PRODUCTION".equals(env)) { warehouseLocation = "Detroit_Main_Hub"; } else { warehouseLocation = "Local_Test_Bay";

```
        }

        // 2. Multi-step initialization of a collection
        supportedCategories.add("Sedan");
        supportedCategories.add("SUV");
        supportedCategories.add("Truck");
    }

    public static void main(String[] args) {
        // Accessing static variables without creating an object
        System.out.println("Location: " + InventoryConfig.warehouseLocation);
        System.out.println("Categories: " + InventoryConfig.supportedCategories);
    }
}
```

Comments

Type	Syntax	Purpose
Single-line	// text	Brief inline notes; anything after // on that line is ignored. Cannot be used mid-statement.
Multi-line	/* text */	Used to explain complex blocks, classes, or multi-line paragraphs. Can span multiple lines
Javadoc	/** text */	Special documentation blocks placed right before class/method declarations The JDK provides a javadoc command-line tool that parses text within / ... */ blocks to automatically output standardized HTML documentation pages. It supports specialized tags to format programmatic traits: • @param: Documents method input parameters. • @return: Describes the value returned by a method. • @since / @version / @author: Specifies framework versions, code changes, and creator identities. • {@code}: Integrates literal code snippets inside documentation text cleanly.

Runtime Polymorphism by data members?

Not possible because method overriding is used to achieve runtime polymorphism and data members cannot be overridden. We can override the member functions but not the data members. Consider the example given below.

```
class Bike{
 int speedlimit=90;
}

class Honda3 extends Bike{
 int speedlimit=150;

 public static void main(String args[]){
  Bike obj=new Honda3();
  System.out.println(obj.speedlimit);//90
   }
```

Why are the objects immutable in java?

Because Java uses the concept of the string literal. Suppose there are five reference variables, all refer to one object "sachin". If one reference variable changes the value of the object, it will be affected by all the reference variables. That is why string objects are immutable in java.

Final Keyword

final variable - A variable declared as final prevents the content of that variable being modified
final method - A method declared as final prevents the user from overriding that method
final class - A class declared as final cannot be extended thus prevents inheritance

Static Method

• A static method is a method which belongs to the class and not to the instance(object)
• A static method can be invoked without the need for creating an instance of a class
• A static method can call only other static methods and cannot call a non-static method from it
• A static method can access static data member and can change the value of it
• A static method cannot refer to this or super keywords in anyway
• Static methods cannot access non static variables while non static functions can access static variables.
• Static methods cannot be overridden because they are not part of the object's state. Rather, they belongs to the class.
• Static method can access static data member and can change the value of it. They can be overloaded.

Static variable

A static variable is one that's associated with a class, not instance (object) of that class. They are initialized only once, at the start of the execution. A single copy to be shared by all instances of the class and it can be accessed directly by the class name and doesn't need any object. One common use of static is to create a constant value that's attached to a class. e.g. public static final EMPLOYER_NAME="Google";

Instance Variable

Declared inside the class but outside any method.

Local Variable

Declared inside the body of a method. Local variables cannot be static.

Memory Usage

Data Type	Value Range	Memory Usage (bits)
byte	-128 to 127	8
short	-32,768 to 32,767	16
int	-2^{31} to $2^{31}-1$	32
long	-2^{63} to $2^{63}-1$ Used when int isn't big enough (needs an 'L' suffix, like 100L).	64
float	Approximately +-3.4e+38. needs an 'f' suffix, like 3.14f because Java thinks all decimals are doubles by default. It stays accurate for about **7 decimal digits**. why 0.7f, are not stored exactly and result in precision errors (e.g., 0.6999999) Some decimals result in infinitely repeating binary sequences. • **0.7 in Binary:** $0.101100110011...$ (repeats infinitely). • **The Problem:** The Mantissa only has **23 bits**. The JVM must truncate the repeating sequence. • **The Result:** When converting that truncated 23-bit sequence back to decimal, the value is slightly less than 0.7 (e.g., 0.699999...).	32
double	Approximately +-1.7e+308 The default for decimals; provides more precision than float.Accurate for about **15–16 decimal digits**	64
char	Unicode characters (0 to 65,535)	16

Component	Size	Purpose
Sign Bit	1 bit	0 for Positive, 1 for Negative.
Exponent	8 bits	Stores the magnitude using a **Bias of 127**.
Mantissa	23 bits	Stores the fractional part (significant digits).

	boolean	true or false	1

BigDecimal

The BigDecimal class provides operations on double numbers for arithmetic, scale handling, rounding, comparison, format conversion and hashing. It can handle very large and very small floating point numbers with great precision but compensating with the time complexity a bit. BigDecimal is an object.

```
// TRAP: This inherits the exact imprecision of the double!
BigDecimal bad = new BigDecimal(0.1); // Prints: 0.1000000000000000055511151231257827021181583404541015625

BigDecimal bd = new BigDecimal("10.5"); (Always use a String in the constructor to keep it precise!).
BigDecimal safePrice = BigDecimal.valueOf( 10.99); // Safe! Equivalent to new BigDecimal("10.99")
BigDecimal safeInt = new BigDecimal(42); // perfectly fine

BigDecimal sum = bd1.add(bd2);
BigDecimal difference = bd1.subtract(bd2);
BigDecimal quotient = bd1.divide(bd2);
BigDecimal product = bd1.multiply(bd2);

assertTrue(bd1.compareTo(bd3) < 0);
assertTrue(bd3.compareTo(bd1) > 0);
assertTrue(bd1.compareTo(bd2) == 0);
assertTrue(bd1.compareTo(bd3) <= 0);
assertTrue(bd1.compareTo(bd2) >= 0);
assertTrue(bd1.compareTo(bd3) != 0);
```

- **Range: Virtually Infinite.** It is limited only by the amount of RAM (memory) your computer has.
- **Precision: Absolute.** It does not use powers of 2 to "approximate" decimals. If you tell it $0.1 + 0.2$, it will give you exactly 0.3, not 0.30000000000000004.
- **Mandatory for money**; never rounds by accident.
- **Usage:** You **must** use this for money or legal calculations.
- Compareto() is used for comparison instead of equals()

Feature	BigInteger	BigDecimal
Primary Use	Massive whole numbers (e.g., cryptography, factorials)	Exact decimal precision (e.g., money, scientific measurements)
Internal Composition	int[] array representing bits	BigInteger (unscaled value) + int (scale)
Operators (+, -)	No, uses methods (.add(), etc.)	No, uses methods (.add(), etc.)
Immutability	Yes	Yes
Comparison	Must use .compareTo()	Must use .compareTo()

Pass-by-Value in Java

When you pass a primitive data type to a method, Java passes a copy of the value. Any changes made to this parameter inside the method do not affect the original value.

```
public static void main(String[] args) {
   int original = 5;
   changePrimitive(original);
   System.out.println(original);  // Outputs: 5
}

static void changePrimitive(int num) {
   num = 10;
}
```

Object References: When you pass an object to a method, Java still passes a copy of the value. The "value" here is the reference to the object in memory, not the actual object. This means you can modify the object's attributes within the method because both the original and the copy of the reference point to the same object.

```
public static void main(String[] args) {
   MyObject obj = new MyObject();
   obj.value = 5;
   changeObject(obj);
   System.out.println(obj.value);  // Outputs: 10
}

static void changeObject(MyObject obj) {
   obj.value = 10;
}

static class MyObject {
   int value;
}
```

HashCode and Equals

If two objects are equal (equals() returns true), their hashCode() must be the same but Two objects with the same hashCode() are not necessarily equal(Hash Collision).

If equals() is overridden without hashCode(), the following issues arise:

- Object may be considered equal but stored in different hash buckets in HashSet or HashMap.
- Leads to data inconsistency, duplicate entries, and unexpected behavior.
- Overriding hashCode() ensures objects that are equal are stored in the same bucket.

In Java, `equals()` and `hashCode()` must be overridden together to maintain the integrity of hash-based collections like `HashMap` and `HashSet`. These collections use `hashCode()` as an initial zip code to route an object to a specific "bucket," and then use `equals()` to find the exact match within that bucket. If you override `equals()` but not `hashCode()`, two logically identical objects will generate different hash codes and end up in different buckets, making it impossible for the collection to find them. Conversely, overriding only `hashCode()` means equal objects will reach the correct bucket but fail the final identity check, leading to broken lookups and duplicate data.

If hashCode() is overridden but equals() is not, objects may end up in the same bucket, but HashMap will treat them as different keys because equality falls back to reference comparison from Object.equals(). This leads to duplicate logical keys and failed lookups.

If equals() is overridden but hashCode() is not, logically equal objects produce different hash codes and are stored in different buckets. Since HashMap uses hashCode() before equals(), the equality check never occurs, resulting in failed lookups and duplicate logical keys.

Two objects stored at different locations can have same hashcode, ex two different string objects created with new keyword having same text john will have same

hashcode.

Regex

Regex is supported by the java.util.regex package, which provides two key classes:

Pattern	Matcher
Used for defining patterns	Used for performing match operations on text using patterns
• Pattern.compile(String regex): Compiles a regular expression into a Pattern object. • Pattern.matches(String regex, CharSequence input): Directly matches a regex pattern against an input string.	• Matcher matcher(CharSequence input): Matches the pattern against the input string. • matcher.find(): Searches for the next subsequence that matches the pattern. • matcher.group(): Returns the matched subsequence.

Character Class	Description
[xyz]	Single Character from x,y or z
[^xyz]	Any characters other than x,y or z
[a-zA-Z]	characters from range a to z or A to Z.
[a-fm-t]]	Union of a to f and m to t.
[a-z && [^bc]]	a to z union with except b and c
[a-z && [^m-p]]	a to z union with except range m to p
X?	X appears once or not
X+	X appears once or more than once
X*	X appears zero or not once
X{n}	X appears n times
X{n,}	X appears n times or more than n
X{n,m}	X appears greater than equal to n times and less than m times.
.	Any character
\d	Any digits, [0-9]
\D	Any non-digit, [^0-9]
\s	Whitespace character, [\t\n\x0B\f\r]
\S	Non-whitespace character, [^\s]
\w	Word character, [a-zA-Z_0-9]
\W	Non-word character, [^\w]
\b	Word boundary
\B	Non -Word boundary

```
public boolean isValidEmail(String email) {
    String regex =  "^[a-zA-Z0-9_+&*-]+(?:\\.[a-zA-Z0-9_+&*-]+)*@(?:[a-zA-Z0-9-]+\\.)+[a-zA-Z]{2,7}$";
    Pattern pattern = Pattern.compile(regex);
    Matcher matcher = pattern.matcher(email);
    return matcher.matches();
  }
```

^	Ensures that the pattern starts matching from the beginning of the string
$	Ensures that the pattern ends matching at the end of the string
[a-zA-Z0-9 _+&*-]+	Any uppercase or lowercase letter (a-z, A-Z) Any digit (0-9) Special characters: _, +, &, *, -. + after] denotes group of such valid chars
(?:\\.[a-zA-Z0-9 _+&*-]+)*	\\. Matches a literal dot (.). The backslash (\) escapes the dot, ensuring it is treated as a literal character rather than a regex wildcard. [a-zA-Z0-9_+&*-]+: After the dot, this part must have at least one valid character (same as defined above). *: Allows this group to appear zero or more times. This makes subdomains in the local part (e.g., user.name.part) optional. (?: ...): A non-capturing group, meaning it groups the content without creating a backreference.
@	Matches the @ character, separating the local part from the domain.
(?:[a-zA-Z0-9-]+ \\.)+	[a-zA-Z0-9-]: Matches a single character that can be: Any uppercase or lowercase letter, Any digit, The hyphen (-). +: Ensures at least one such character exists. \\.: Matches a literal dot (.). (?: ...)+: The group repeats one or more times, ensuring there is at least one domain level (e.g., example. in example.com).

Lombok

Annotation	Description
@Getter	Generates getter methods for all fields in the class.
@Setter	Generates setter methods for all fields in the class.
@ToString	Generates a toString method including all class fields by default. Customizable to exclude certain fields.
@EqualsAndHashCode	Generates equals and hashCode methods based on the fields of the class. Customizable to exclude certain fields.
@NoArgsConstructor	Generates a no-argument constructor for the class.

@AllArgsConstructor	Generates a constructor with one parameter for each field in the class.
@RequiredArgsConstructor	Generates a constructor for final fields and fields marked with @NonNull.
@Data	Combines @Getter, @Setter, @ToString, @EqualsAndHashCode, and @RequiredArgsConstructor annotations.
@Value	Used for immutable classes; generates getters, toString, equals, hashCode, and a constructor.
@Builder	Generates the builder pattern for object creation. Customizable to include specific fields only.
@Singular	Used with @Builder to generate methods that add a single element to a collection (e.g., List, Set).
@NonNull	Generates a null-check for fields annotated with it. Commonly used in constructors and methods.
@Delegate	Generates delegation methods for a field. The methods of the delegate type are added to the owning class.
@Slf4j	Generates a Slf4j logger field in the class. Other logging frameworks are supported as well (e.g., @Log4j, @Log).
@Cleanup	Ensures a resource (e.g., a stream) is cleaned up by closing it after usage.
@SneakyThrows	Allows throwing checked exceptions without explicitly declaring them in the method signature.
@Accessors	Customizes the naming conventions for generated getters and setters (e.g., fluent style).
@FieldDefaults	Sets default visibility and other modifiers for fields (e.g., private, final).
@With	Generates a "with" method for creating a copy of an immutable object with one field changed.
@SuperBuilder	Similar to @Builder, but supports inheritance by generating builders for superclasses and subclasses.
@Getter(lazy = true)	Generates a lazy-initialized getter for a field, initializing it only on the first access.
@NoArgsConstructor(force = true)	Forces generation of a no-argument constructor, initializing final fields with default values.
@RequiredArgsConstructor(staticName = "of")	Generates a constructor for final fields and fields marked with @NonNull, and provides a static method named of to create instances.
	Works by plugging into the build process and auto-generates methods at compile time: 1. You write a clean POJO with **just fields** and a few annotations. 2. On compilation, Lombok injects getters/setters/constructors/etc. into the **.class (bytecode)**. 3. Methods are **not visible in your source code** but available in compiled class → JVM can use them normally.

Clean Code

Naming	• Give meaningful and short names to variable function and classes so that another person doesn't have to read the code to understand what is happening. • Variables/Constants: Use nouns or short phrases with adjectives. e.g userData, isValid • Functions: Verbs. eg sendData(),getUser(),saveUser() • Classes: Nouns. eg UserBody, RequestBody • Don't include redundant info in names like userWithNameAndAge • Avoid Slangs like user.diePlease() • Be Consistent with naming like if you use getUser then use getProduct, don't use fetchProduct()
Ordering Functions	If one function calls another, the next function should be below the 1st function to improve readability.
Definition	• Working and calling of the function should be easy/readable. • What Matters • Number and order of arguments • Length of the function • Rules • Minimise number of parameters • Try to use max 2 parameters • Use Maps for more than 2 parameters so that order of parameters doesn't matter • It should be small: A function should do one thing and outsource others

This function takes 2 arguments

```
function saveUser(email, password) {
  const user = {
    id: Math.random().toString(),
    email: email,
    password: password,
  };

  db.insert('users', user);
}
```

Create another function

```
function saveUser(user) {
  db.insert('users', user);
}
```

Create A class

```
class User {
  constructor(email, password) {
    this.email = email;
    this.password = password;
    this.id = Math.random().toString();
  }

  save() {
    db.insert('users', this);
  }
}

const user = new User('test@test.com', 'testers');
user.save();
```

Refactor	• Extract codes which work on same functionality • Extract code which require more interpretation than surrounding code
	Follow SRP and DRY

One Public Class per file

The restriction of having only one public class per Java file is a design choice to assist the **Java Virtual Machine (JVM)** in efficiently locating the program's entry point. By forcing the file name to match the single public class, the JVM knows exactly which .class file to look for (e.g., B.java results in B.class) to find the main method.

```
// File must be named: Employee.java
public class Employee {
   public static void main(String[] args) {
      // Entry point accessible to JVM via Employee.main()
   }
}

class Manager {
   // Non-public classes can exist but cannot dictate file name
}
```

Reflection API

Reflection is an API which allows you to inspect and modify the runtime behavior of applications. It allows you to analyze a class's structure (fields, methods, constructors) even if they are private, and even change values of final variables.

Cat.java

```
public class Cat {
    private final String name; // Private and Final!
    private int age;

    public Cat(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() { return name; }

    public void meow() {
        System.out.println("Meow!");
    }

    private void privateMethod() {
        System.out.println("This is a private method.");
    }

    public static void publicStaticMethod() {
        System.out.println("I am a public static method.");
    }

   private static void privateStaticMethod() {
        System.out.println("I am a private static method.");
    }

}
```

Main.java

```
import java.lang.reflect.Field;
import java.lang.reflect.Method;

public class ReflectionDemo {
    public static void main(String[] args) throws Exception {
        Cat myCat = new Cat("Stella", 6);

        // 1. Get the Class object (The window into reflection)
        Class<?> catClass = myCat.getClass();

        // 2. Accessing and Modifying Private Final Fields
        Field[] fields = catClass.getDeclaredFields();
        for (Field field : fields) {
            if (field.getName().equals("name")) {
                // Bypass the "private" and "final" security
                field.setAccessible(true);

                // Change the value on our specific instance
                field.set(myCat, "Jimmy McGill");
            }
        }
        System.out.println("New Name: " + myCat.getName());

        // 3. Invoking Private Methods
        Method[] methods = catClass.getDeclaredMethods();
        for (Method method : methods) {
            if (method.getName().equals("privateMethod")) {
                method.setAccessible(true);
                method.invoke(myCat); // Executes the method
            }

            // 4. Invoking Static Methods
            if (method.getName().equals("publicStaticMethod")) {
                // Pass 'null' because static methods don't need an object instance
                method.invoke(null);
            }
        }
    }
}
```

setAccessible(true)	This is the most critical part of your notes. By default, reflection respects Java's encapsulation. Calling setAccessible(tru e) tells the JVM to ignore the private modifier for that specific field or method session.
Usage	Spring and Hibernate use it to inject data into your objects without you writing Setters for everything.

Parameter vs Args

Concept	Definition
Parameter	Variable in method definition
Argument	Actual value passed during method call

Method Signature

Element	Considered in Signature
Method name	Yes
Number of parameters	Yes
Data types of parameters	Yes
Order of parameters	Yes

Ignored Element

Modifier (public, private, etc.)

Return type (int, void, etc.)

Parameter names (num1, num2)

throws clause

Null	In Java, null is a special literal that represents the absence of an object reference. When a reference variable is assigned null, it means it does not point to any object in memory. This is applicable only to reference types like classes, arrays, and interfaces, not to primitive types. Attempting to access fields or methods using a null reference results in a NullPointerException, so developers typically perform null checks before using such variables.
Debugging	• **Step Over (F8):** Moves to the next line in the current method. Use this to watch the loop iterate. • **Step Into (F7):** If you are on a method call, this "goes inside" that method. • **Step Out**: Finish the current method and jump back up to the caller. • **Resume Program (F9):** Jumps to the next breakpoint or finishes the program.
Logger	`public class LoggerDemo {` `    private static Logger logger = Logger.getLogger(LoggerDemo.class.getName());` `    public static void main(String[] args) {` `        // set log level to SEVERE, default level info` `        logger.setLevel(Level.SEVERE);` `        logger.info("This is info level logging");` `        logger.log(Level.WARNING, "This is warning level logging");` `        logger.log(Level.SEVERE, "This is severe level logging");` `        System.out.println("Hello using System.out.println");` `    }` `}`
Fail-Fast vs Fail-Safe	**Fail-Fast**: Throws ConcurrentModificationException if the collection is modified during iteration. Examples • ArrayList • HashMap • HashSet **Fail-Safe**: Iterates over a copy of the collection. No exception. Examples • CopyOnWriteArrayList • ConcurrentHashMap

Program Notes

04 March 2026 17:12

<table>
<tr><td>Notes</td><td>
<table>
<tr><th>Program</th><th>Notes</th></tr>
<tr><td>ContainsDuplicate</td><td>• Sort the array and then nums[i] == nums[i + 1]
• Use hashset to add elements, if add returns false, return true.
• Else use i,j loops to Brute Force</td></tr>
<tr><td>LargestUniqueNumber</td><td></td></tr>
</table>
</td></tr>
<tr><td>Char Array to String</td><td>String key = new String(chars);</td></tr>
<tr><td rowspan="6">Hashmap</td><td>• anagramMap.computeIfAbsent(key, k -> new ArrayList<>());
• charCountMap.put(c, charCountMap.getOrDefault(c, 0) + 1);</td></tr>
<tr><td>List<String> names = List.of("Arnav", "Aadil", "Rohit", "Aakash", "Kohli");</td></tr>
<tr><td>List<String> result = names.stream().filter(s -> s.length() > 5).map(String::toUpperCase).sorted().toList();</td></tr>
<tr><td>Stream<String> newStream = Streams.concat(names.stream(), result.stream());</td></tr>
<tr><td>for (Map.Entry<String, Integer> entry : frequencyMap.entrySet()) {
String word = entry.getKey();
int frequency = entry.getValue();

if (frequency > maxFrequency) {
secondMaxFrequency = maxFrequency;
secondMostFrequent = word;
maxFrequency = frequency;
} else if (frequency > secondMaxFrequency && frequency < maxFrequency) {
secondMaxFrequency = frequency;
secondMostFrequent = word;
}
}</td></tr>
<tr><td>case-insensitive search</td><td>str.toLowerCase().contains("world");</td></tr>
<tr><td>find multiple occurrences</td><td>int index = str.indexOf("world");

while (index != -1) {
System.out.println(index);
index = str.indexOf("world", index + 1);
}</td></tr>
<tr><td>Arrays.copyOf(arr, n)</td><td>Copies n elements to new array</td></tr>
<tr><td>Arrays.toString(str)</td><td>Converts an array to a readable string format</td></tr>
<tr><td>Index of a char in String</td><td>String word="dd";
System.out.println(word.indexOf('d'));//0
System.out.println(word.lastIndexOf('d'));//1
If both are same then digit is not repeated in string, else repeated</td></tr>
<tr><td>SubString Function</td><td>String str = "Hello, World!";
String result = str.substring(7);
System.out.println(result); // Output: "World!"

String str = "Hello, World!";
String result = str.substring(7, 12);
System.out.println(result); // Output: "World"

beginIndex is the starting index (inclusive) of the substring.
endIndex is the ending index (exclusive) of the substring</td></tr>
<tr><td>Arrays.sort(arrayA);</td><td>Sorts the elements of an array in ascending order

Integer[] arrayA = {5, 3, 8, 1, 2}; // Note: Changed to Integer[]
Arrays.sort(arrayA, Collections.reverseOrder()); //Comparators in Java require Objects
System.out.println(Arrays.toString(arrayA)); // Output: [8, 5, 3, 2, 1]</td></tr>
<tr><td>Arrays.equals(arrayA, arrayB);</td><td>Used to compare two arrays for equality. It checks if both arrays contain the same elements in the same order

Variation:
ArrayList<Integer> list1 = new ArrayList<>(Arrays.asList(5, 2, 7, 1, 4));
boolean isEqual1 = list1.equals(list2);</td></tr>
<tr><td>Char array to String</td><td>str = new String(ch);</td></tr>
<tr><td>Array Fill</td><td>• Arrays.fill(array, value): Populates all or a targeted range of an array with a uniform value.
• Arrays.setAll(array, lambda): (Java 8+) Populates an array dynamically using a generator lambda function (e.g., Arrays.setAll(indices, i -> i);).</td></tr>
</table>

Multi-Threading

30 March 2025 10:59

<table>
<tr><td>Concurrency</td><td>• The illusion of multiple tasks running simultaneously (even on a single core).
• Achieved by rapidly switching between threads.</td></tr>
<tr><td>Parallelism</td><td>• True simultaneous execution of tasks, possible only with multi-core CPUs.
• Boosts performance by distributing workload across multiple cores.</td></tr>
<tr><td>Process</td><td>• An instance of a running application. Has it's Own memory space and is Heavyweight.
• Each process is isolated from others.
• Contains:
○ Program code
○ Heap memory
○ Threads
○ File handles
○ Network sockets
○ OS resources
○ Security information
• Processes are isolated from each other, making context switches between processes slower.</td></tr>
<tr><td>Thread</td><td>• Unit of execution inside process - Shares heap with other threads. Each thread has its own stack
• Each thread has:
• Stack – Stores local variables and function calls.
• Instruction Pointer – Tracks the next instruction to execute.
• Threads:
• Consume fewer resources since they share code, heap, and file references.
• Context switches between threads in the same process are faster.

<table><tr><th>Feature</th><th>Platform Thread</th><th>Virtual Thread</th></tr><tr><td>Backed by OS thread</td><td>Yes</td><td>No (uses carrier threads internally)</td></tr><tr><td>Default type</td><td>User thread</td><td>Daemon thread</td></tr><tr><td>Default name</td><td>Thread-0, Thread-1...</td><td>No default name</td></tr><tr><td>Builder</td><td>Thread.ofPlatform()</td><td>Thread.ofVirtual()</td></tr><tr><td>Dedicated stack</td><td>Yes</td><td>No fixed native stack</td></tr><tr><td>Scalability</td><td>Thousands</td><td>Hundreds of thousands to millions</td></tr><tr><td>Resource usage</td><td>High</td><td>Very low</td></tr><tr><td>Mental Model</td><td>Platform Thread
↓
1 Java Thread
↓
1 OS Thread
↓
Limited scalability</td><td>Virtual Thread
↓
1 Java Thread
↓
Managed by JVM
↓
Runs on carrier threads
↓
Massive scalability</td></tr></table></td></tr>
<tr><td>Virtual Thread</td><td>• Lightweight JVM-managed thread.
• Not mapped 1:1 to an OS thread.
• Represents a task, not a native execution thread.
• Created as a normal Java object on the Heap.
• Introduced to efficiently handle blocking I/O.</td></tr>
<tr><td>CPU's Thread Execution</td><td>CPU executes one instruction stream per core.

Example:
Time 1 -> Thread A
Time 2 -> Thread B
Time 3 -> Thread C

This switching is called Context Switching.

Problems: - Register save/restore - Cache invalidation - Scheduler overhead

Too many threads cause Thrashing.</td></tr>
<tr><td>Context Switch(Thread Scheduler)</td><td>Definition: The process of stopping one thread, saving its state, switching to another thread, and restoring its state.
• Why is it Needed?
○ Multiple threads compete for limited CPU cores.
○ Even on multi-core CPUs, there are usually more threads than cores, requiring the OS to manage thread execution.

Why Are Context Switches Costly?
• Threads rely on resources like:
○ CPU registers
○ CPU caches
○ Kernel resources in memory
• During a context switch:
○ The current thread’s state (registers, cache data, etc.) is stored.
○ The next thread’s state is loaded back into the CPU.

Impact: Frequent context switches reduce productive work and increase overhead.</td></tr>
<tr><td>Thrashing</td><td>Occurs when too many threads cause excessive context switching, leaving little CPU time for actual task execution.</td></tr>
<tr><td>Time Slicing</td><td>• Time slicing is a CPU scheduling mechanism in which the CPU time is divided into small units called time slices (or time quanta).
• Each thread gets a fixed time slice to execute before the CPU switches to another thread.
• If a thread does not complete its execution within the allotted time slice, it is preempted and placed back in the queue.
• Java does not provide direct control over time slicing, as it depends on the OS and JVM implementation</td></tr>
<tr><td>Thread Lifecycle</td><td>• NEW → Before start()</td></tr>
</table>

- Active
  - RUNNABLE → Ready to run but waiting for CPU
  - Running
- BLOCKED → Waiting for a lock
  - WAITING → Indefinite waiting (needs notify())
  - TIMED_WAITING → Waiting for a fixed time (sleep(), wait(time))
- TERMINATED → Completed execution or stopped

Creating Threads

Extending Thread	`class MyThread extends Thread {` `  public void run() {` `  System.out.println("Thread running");` `  }` `}` `new MyThread().start();`
Implementing Runnable	`Runnable task = () -> System.out.println("Running");` `new Thread(task).start();`
Using Callable	`Callable<Integer> task = () -> 100;` `ExecutorService es = Executors.newSingleThreadExecutor();` `Future<Integer> result = es.submit(task);` `System.out.println(result.get());` `es.shutdown();`
Lambda	`new Thread(() -> work()).start();`
Preferred Approach	`ExecutorService executor = Executors.newFixedThreadPool(5);` `executor.submit(() -> System.out.println("Task"));` `executor.shutdown();`

Runnable vs Callable

Runnable: - returns nothing - cannot throw checked exception

Callable: - returns value - can throw checked exception

Callable c = () -> 100;

Future f = executor.submit(c);

- Callable<T> is an **interface** in java.util.concurrent used to represent **asynchronous tasks that return a result**.
- Unlike Runnable, Callable **can return a value** and **throw checked exceptions**

```
import java.util.concurrent.*;

public class CallableExample {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(2);

        // Submitting a Callable task
        Future<Integer> futureResult = executor.submit(() -> {
            System.out.println("Executing Callable Task...");
            Thread.sleep(2000);
            return 42;  // Returning a value
        });

        try {
            System.out.println("Result: " + futureResult.get());  // Blocking call
        } catch (InterruptedException | ExecutionException e) {
            e.printStackTrace();
        }

        executor.shutdown();
    }
}
```

Join()

The join() method in Java makes the calling thread wait until the specified thread completes its execution. It is used to ens ure sequential execution when multiple threads are running. When a thread calls t.join(), it pauses execution until thread t completes.

- **join**() → Waits indefinitely until the thread finishes.
- **join**(**millis**) → Waits for a maximum of millis milliseconds, then continues.

User Vs Daemon Thread

Feature	User Thread	Daemon Thread
Definition	A thread that **executes user-defined tasks** and keeps the JVM running until it completes.	A background thread that runs **supportive tasks** and automatically stops when all user threads finish.
JVM Behavior	JVM **waits** for all user threads to complete before shutting down.	JVM **terminates daemon threads automatically** when no user threads remain.
Purpose	Used for main application logic (e.g., handling requests, running tests).	Used for background tasks (e.g., garbage collection, logging, monitoring).
Lifecycle	Must complete its execution before JVM shuts down.	JVM **does not wait** for daemon threads to finish; they may be stopped abruptly.
Default Type	By default, all threads are **User Threads** unless explicitly set as Daemon.	Must be explicitly marked as **Daemon** using setDaemon(true).

Thread Scheduler

The Thread Scheduler in Java is part of the JVM and OS that decides which thread to execute next from the ready queue. It is non-deterministic, meaning we cannot guarantee the exact order of execution.

It uses a mix of:
- **Preemptive Scheduling** (higher priority threads may interrupt lower ones)
- **Time-Sliced** (Round-Robin) Scheduling (each thread gets a small time slice)

<table>
<tr><td>Thread Priority</td><td>Each thread in Java has a priority, which helps the scheduler decide which thread to run first.

Thread priority values range from 1 to 10, where:
• Thread.MIN_PRIORITY = 1
• Thread.NORM_PRIORITY = 5 (default)
• Thread.MAX_PRIORITY = 10

However, priority is just a hint—it does not guarantee execution order. Thread Scheduler stores threads in a queue and same priority are FIFO.

t1.setPriority(Thread.MIN_PRIORITY); // 1
t2.setPriority(Thread.NORM_PRIORITY); // 5
t3.setPriority(Thread.MAX_PRIORITY); // 10

System.out.println(Thread.currentThread().getName());
System.out.println(Thread.currentThread().getPriority());
Thread.currentThread().setPriority(Thread.MAX_PRIORITY);
System.out.println(Thread.currentThread().getPriority());</td></tr>
<tr><td>Synchronization</td><td>synchronized(lock){

updateReport();

}

Guarantees:

- one thread enters
- others wait

Automation examples:

- report writing
- screenshot storage
- file writing
- shared counters</td></tr>
<tr><td>Volatile</td><td>The volatile keyword is a visibility modifier. It ensures that changes made to a variable by one thread are immediately visible to all other threads. It’s used when a variable that is written to by only one thread and read by many threads (like a shutdown flag or status indicator), where you just need everyone to see the change instantly.</td></tr>
<tr><td>Atomic Classes</td><td>Atomic classes (like AtomicInteger, AtomicBoolean, AtomicReference, etc.) provide atomicity without the heavy overhead of standard synchronization (synchronized blocks or locks). It's used when a variable that is being read and modified by multiple threads simultaneously (like counters, sequence generators, or statistical accumulators), and operations depend on the current value.

How they work: They use low-level CPU instructions called CAS (Compare-And-Swap). Instead of locking a variable, a CAS operation says: "Here is the old value I read, and here is the new value I want to set. If the current value is still the old value, update it. If not, fail and try a gain."
Because this happens at the hardware level, it is incredibly fast and non-blocking.

The Catch: While highly efficient for single variables, they cannot be used to combine multiple operations atomically (e.g., you can't make an atomic update across two different AtomicInteger variables together).</td></tr>
<tr><td>Compare-and-Swap (CAS)</td><td>Compare-and-Swap (CAS) is a foundational atomic CPU instruction used in multithreading to synchronize access to shared data. It operates optimistically: a thread attempts to update a variable by providing its expected old value and a new value; if the memory still holds the expected old value, the swap succeeds, otherwise, it fails</td></tr>
<tr><td>wait() and notify()</td><td>wait() and notify() are used for inter-thread communication, allowing threads to coordinate execution. They are methods of the Object class and must be called within a synchronized block.

How wait() Works
• A thread calls wait() inside a synchronized block, releasing the lock and entering a waiting state.
• The thread remains blocked until another thread calls notify() or notifyAll().

How notify() Works
• A thread calls notify() to wake up one waiting thread.
• The woken thread reacquires the lock and resumes execution after the notify thread has completed its execution</td></tr>
<tr><td>ExecutorService</td><td>ExecutorService is a framework in Java that manages and controls thread execution efficiently. It belongs to the java.util.concurrent package and provides a higher-level replacement for manually creating and managing threads. Instead of creating and starting threads manually, we use Execu torService to handle thread pooling, task submission, and lifecycle management.

Problems with Traditional Threads
• High Overhead → Creating a new thread for every task is expensive.
• Difficult to Manage → No built-in way to control thread execution.
• No Thread Reuse → Once a thread finishes execution, it cannot be reused.

How ExecutorService Solves These Problems
• Thread Pooling → Reuses a fixed number of threads instead of creating new ones.
• Better Performance → Reduces thread creation overhead.
• Task Queuing → Handles multiple tasks efficiently.
• Graceful Shutdown → Provides methods to manage lifecycle (shutdown(), awaitTermination()).

<table>
<tr><td>Fixed Thread Pool (Best for CPU-Intensive Tasks)</td><td>Best for scenarios where the number of threads is known in advance.

<pre>
public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(3);

        for (int i = 1; i <= 5; i++) {
            final int taskId = i;
            executor.submit(() -> {
                System.out.println("Executing Task " + taskId + " by " +
Thread.currentThread().getName());
            });
        }
        executor.shutdown();
    }
</pre>
</td></tr>
</table>
</td></tr>
</table>

<table>
<tr><td></td><td>
<table>
<tr><td>Cached Thread Pool (Best for Short-Lived Tasks)</td><td>• Creates new threads as needed and reuses them when possible.
• Best for tasks with unpredictable execution times.
• Thread is killed if it is idle for more than 60 seconds.

ExecutorService executor = Executors.newCachedThreadPool();</td></tr>
<tr><td>Single Thread Executor (Best for Sequential Execution)</td><td>• Executes tasks one by one in a single thread.
• Ensures task ordering.

ExecutorService executor = Executors.newSingleThreadExecutor();</td></tr>
<tr><td>Scheduled Thread Pool (Best for Delayed or Repeated Tasks)</td><td>ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(2);

// Schedule task with a delay
scheduler.schedule(() -> System.out.println("Task executed after 2 seconds"), 2, TimeUnit.SECONDS);

// Schedule a repeating task
scheduler.scheduleAtFixedRate(() -> System.out.println("Repeating task"), 1, 3, TimeUnit.SECONDS);

Use Case: Periodic background tasks, Auto-save functionality, Monitoring.</td></tr>
</table>
</td></tr>
<tr><td>Internal Working</td><td>When a task is submitted (submit() or execute()), the following steps occur:
1. Task is added to a queue (if all worker threads are busy).
2. Worker threads pick tasks from the queue and execute them.
3. If queue(LinkedBlockingQueue) is full, a new thread is created (up to the max limit in ThreadPoolExecutor).
4. If max threads are reached, the task is either rejected or handled based on the rejection policy.
5. Threads are reused instead of creating new ones for every task.
6. When shutting down, ExecutorService ensures all running tasks complete before termination.</td></tr>
<tr><td>SingleThreadExecutor</td><td>public class SingleThreadExecutorDemo {
public static void main(String[] args) {
try (ExecutorService service = Executors.newSingleThreadExecutor()) {
for (int i = 0; i < 5; i++) {
service.submit(new Task(i));
}
}
}
}

class Task implements Runnable {
private final int taskId;

public Task(int taskId) {
this.taskId = taskId;
}

@Override
public void run() {
System.out.println("Task with ID : " + taskId + " being executed by thread : " + Thread.currentThread().getName());
try {
Thread.sleep(500);
} catch (InterruptedException e) {
throw new RuntimeException(e);
}
}
}</td></tr>
<tr><td>FixedThreadPool</td><td>public class FixedThreadPoolDemo {
public static void main(String[] args) {
try (ExecutorService executor = Executors.newFixedThreadPool(2)) {
for (int i = 0; i < 7; i++) {
executor.execute(new Work(i + 1));
}
}
}
}

class Work implements Runnable {
private final int workId;
public Work(int workId) {
this.workId = workId;
}

@Override
public void run() {
System.out.println("Task with ID : " + workId + " being executed by thread : " + Thread.currentThread().getName());
try {
Thread.sleep(700);
} catch (InterruptedException e) {
throw new RuntimeException(e);
}
}
}</td></tr>
<tr><td>ThreadPoolExecutor()</td><td>ThreadPoolExecutor is the core implementation behind ExecutorService. It provides full control over:
1. Number of threads (core and max)
2. Task queuing mechanism
3. Thread lifecycle management
4. Task rejection handling

It is part of java.util.concurrent and is used when built-in Executors (e.g., Executors.newFixedThreadPool()) don’t provide enough customization.

static {
int nCpu = Runtime.getRuntime().availableProcessors();
LOGGER.info("No. of CPU in the m/c: " + nCpu);</td></tr>
</table>

```
    TPEXECUTOR = new ThreadPoolExecutor(10,
                            //Runtime.getRuntime().availableProcessors(),
                            10,
                            50000L,
                            TimeUnit.MILLISECONDS,
                            new LinkedBlockingQueue<Runnable>());
}
```

Java-MR

11 April 2026 11:07

<table>
<tr><td>Java Vendors</td><td>⇒ OpenJDK: Java SourceCode + language specifications. Release new versions every 6 months. Cannot be installed. Comes with TCK (Technology Compatibility Kit) to ensure : All Java implementations behave the same and No vendor breaks Java standards
⇒ Vendors (Oracle, Amazon, Azul, Adoptium): Takes OpenJDK source and adds Performance optimizations, Security patches, Stability improvements. They also provide installable binaries.
⇒ Output of the code will remain same across vendors, performance can differ.</td></tr>
<tr><td>Keywords</td><td>In Java, a keyword is a reserved word that has a specific meaning and purpose in the programming language. These keywords cannot be used as identifiers such as variable names or method names in the code because they are already predefined with a particular function in the Java language.

Java syntax requires all code, including keywords, to be case-sensitive. Therefore, public with all lowercase letters is a keyword, but Public with a capital letter is not a keyword and has a completely different meaning.

The words true, false, and null might seem like keywords, but they are not. Instead, true and false function as Boolean literals, while null serves as a null literal. However, despite not being keywords, true, false, and null still cannot be used as identifiers in Java.

• Reserved words with predefined meaning in Java.
• Defined by the Java language specification.
• Keywords are always lowercase
• Cannot be used as:
• Variable names
• Method names
• Class names
• Example: if, try, catch
• thIS is not a keyword as IS are in uppercase.
• goto and const are reserved but unused keywords
• true, false and null are literals and not keywords. These cannot be used as identifiers.</td></tr>
<tr><td>Data Types</td><td>
<table>
<tr><th>Type</th><th>Purpose</th></tr>
<tr><td>Primitive</td><td>• Built-in, low-level data types
• Store actual values directly in memory
• No object overhead
• Stores simple values like int, boolean
<table>
<tr><th>Type</th><th>Default</th><th>Size</th><th>range</th></tr>
<tr><td>Boolean</td><td>false</td><td>1 bit</td><td>NA</td></tr>
<tr><td>char</td><td>\u0000 (empty Value)</td><td>16 bits=2 Bytes</td><td>\u0000 to \uFFFF
Supports ~65,000+ characters (2^16)</td></tr>
<tr><td>byte</td><td>0</td><td>8 bits=1 Bytes</td><td>-128 to 127</td></tr>
<tr><td>short</td><td>0</td><td>16 bits=2 Bytes</td><td>-32768 to 32767</td></tr>
<tr><td>int</td><td>0</td><td>32 bits=4 Bytes</td><td>-2147483648 to -2147483647</td></tr>
<tr><td>long</td><td>0</td><td>64 bits=8 Bytes</td><td>9223372036854775808 to 9223372036854775807
Add L at the end of variable</td></tr>
<tr><td>float</td><td>0.0</td><td>32 bits=4 Bytes</td><td>1.4E-45 to 3.4028235E38
~7 decimal digits, need to add F suffix</td></tr>
<tr><td>double</td><td>0.0</td><td>64 bits=8 Bytes</td><td>4.9E-324 to 1.7976931348623157E308
~15 decimal digits
this is the default</td></tr>
</table>
</td></tr>
<tr><td>Reference</td><td>Stores complex objects like String, Array, Object</td></tr>
</table>
</td></tr>
<tr><td>Naming Convention</td><td>• Variable: Camel Case, cannot start with numbers. Can begin with_ or $
• Classes: Pascal case
• Function: Camel Case</td></tr>
<tr><td>camel Case</td><td>numberOfPlayers- Java and JS</td></tr>
<tr><td>pascal Case</td><td>NumbersOfPlayers- C# and .NET</td></tr>
<tr><td>snake Case</td><td>number_of_players- Python</td></tr>
<tr><td>kebab Case</td><td>number-of-players- URL and CSS</td></tr>
<tr><td>boolean</td><td>isValid or hasChildren</td></tr>
<tr><td>Escape Sequence</td><td>• Backspace: \b
• Tab: \t
• Newline: \n
• Double quote: \"
• Single quote: \'
• Backslash: \\</td></tr>
<tr><td>Overflow and Underflow</td><td>Overflow occurs when an arithmetic operation creates a value larger than the maximum capacity of the data type

int max = Integer.MAX_VALUE; // 2147483647
int overflow = max + 1;
System.out.println(overflow); // Output: -2147483648

Underflow is the opposite. It happens when a value becomes smaller than the minimum capacity

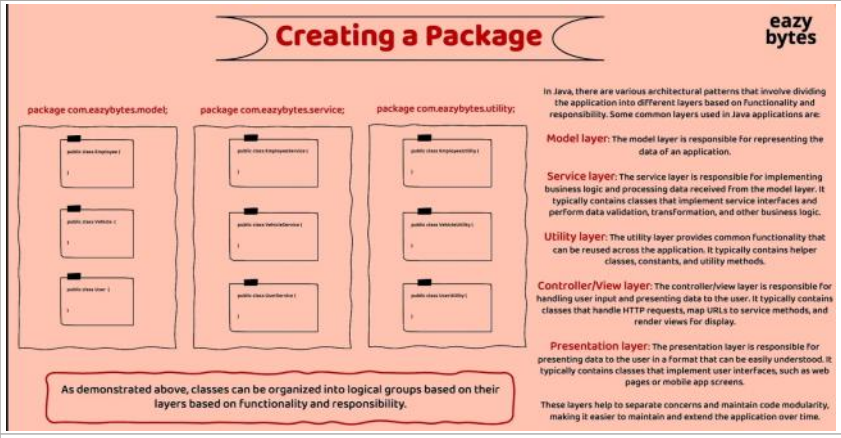
int min = Integer.MIN_VALUE; // -2147483648
int underflow = min - 1;
System.out.println(underflow); // Output: 2147483647</td></tr>
<tr><td>Float and Double</td><td>Double vs Float should be based on precision, not number size.

<table>
<tr><th>Scenario</th><th>Result</th></tr>
<tr><td>1.5 / 0.0</td><td>Infinity</td></tr>
<tr><td>1.5 / -0.0</td><td>-Infinity</td></tr>
<tr><td>0.0 / 0.0</td><td>NaN (Not a Number)</td></tr>
</table>
</td></tr>
</table>

<table>
<tr><td></td><td>
• Float.MIN_VALUE, Float.MAX_VALUE
• Double.MIN_VALUE, Double.MAX_VALUE
• Float.POSITIVE_INFINITY, Float.NEGATIVE_INFINITY, Float.NaN. Same available in Double

Comparing Float and Double

float a = 3.14f;
double b = 3.14;
a == b // false
</td></tr>
<tr><td>Storing Large Numbers</td><td>int num=1_00_00_000;</td></tr>
<tr><td>Number Systems</td><td>
<table>
<tr><th>Format</th><th>Base</th><th>Allowed Digits</th><th>Prefix in Java</th><th>Example</th><th>Notes</th></tr>
<tr><td>Decimal</td><td>10</td><td>0–9</td><td>No prefix</td><td>25</td><td>Default format</td></tr>
<tr><td>Octal</td><td>8</td><td>0–7</td><td>0</td><td>031</td><td>Must start with 0</td></tr>
<tr><td>Hexadecimal</td><td>16</td><td>0–9, A–F</td><td>0x / 0X</td><td>0x453F</td><td>Case-insensitive</td></tr>
<tr><td>Binary</td><td>2</td><td>0, 1</td><td>0b / 0B</td><td>0b011</td><td>Case-insensitive</td></tr>
</table>
</td></tr>
<tr><td>String</td><td>It’s non primitive data type because we cannot fix the size of string</td></tr>
<tr><td>Ternary</td><td>
score >= 90 -> "Grandmaster"
score >= 80 -> "Master"
score >= 70 -> "Expert"
score >= 60 -> "Intermediate"
score < 60 -> "Beginner"

return (score >= 90) ? "Grandmaster" :
(score >= 80) ? "Master" :
(score >= 70) ? "Expert" :
(score >= 60) ? "Intermediate" : "Beginner";
</td></tr>
<tr><td>Sum of Byte in Int</td><td>In Java, when you add two byte variables together, the result is automatically promoted to an int</td></tr>
<tr><td>MVC</td><td>


</td></tr>
<tr><td></td><td></td></tr>
</table>